Risk Methods — Gold Reference

Complete reference for VaR/ES, backtesting, sensitivities, P&L attribution, vol/cov estimation, stress, and capital frameworks. Anchored to the drills' verified numbers (`hist_var_drill.py`, `var_methods_drill.py`, `pnl_attribution_drill.py`).

Each entry: **(1) what it is → (2) how it's calculated → (3) formulas with I/O meaning → (4) toy example → (5) canonical Python → (6) pattern to remember**.

Contents

Universal symbols

```
P&L           random profit-and-loss over horizon h (today → today+h)
α             confidence level (0.99 = 99%); tail mass = 1 − α
V₀            portfolio value today (the "base")
V(scenario)   portfolio value after applying a market-state perturbation
s             sensitivity vector (∂V/∂factor_i) — DV01, CS01, vega, FX-Δ, …
Σ             factor covariance matrix (sized #factors × #factors)
F             matrix of historical factor changes (rows = days, cols = factors)
z(α)          standard-normal quantile, e.g. z(0.99) = 2.326, z(0.995) = 2.576
N(·), φ(·)    std-normal CDF, PDF
```

Sign convention used throughout: **loss is a POSITIVE number** (VaR, ES, drawdown). P&L is negative when you lose.

DV01 convention used throughout this document: `s = V(bumped) − V(base)`. Under this convention, **a long bond's DV01 is NEGATIVE** (rates up → price down → V_bumped < V_base). **Bloomberg and most risk systems quote DV01 with the OPPOSITE sign** (loss-given-up; long bond DV01 is positive). If pulling a vendor DV01 into this cheatsheet's `s · Δr` Taylor formulae, **flip the sign**.

Horizon scaling under iid-normal returns: a 1-day VaR scales to an h-day VaR as `VaR_h = √h · VaR_1`. This breaks under: autocorrelated returns, vol clustering (use FHS), options with non-trivial gamma over the horizon (use full-reval MC). All of FRTB IMA's per-factor liquidity-horizon (LH) scaling rests on this rule applied to **non-overlapping** horizons (§37).

Reference portfolio used in every example

Anchored to `extension_drill2.py` capstone (used by `var_methods_drill.py` and `pnl_attribution_drill.py`):

```
PF = Portfolio([
    Bond((1..5),(4,4,4,4,104),150bp),  IRS((1..5),1.0,0.042,100),
    CDS((1..5),1.0,100,120,100),       TRS(Bond,90),
    ForeignBond((1..5),(3,3,3,3,103),100,1.25),
    Swaption(2.0,(3,4,5),1.0,0.042,0.20,100),
    Repo(100,0.043,0.5), SecLending(100,20,(1,2,3),1.0), Loan(100,0.05,5,200),
    FXSpot(80,1.25), FXOption(1.0,1.30,0.12,1.25,100)])

# 250-day factor history (rate_bp, spread_bp, d_vol, fx_ret):
F[i] = [8 sin(0.3 i) + ((i%11)−5),
        5 cos(0.2 i) + ((i%7)−3),
        0.005 sin(0.4 i) + 0.001 ((i%3)−1),
        0.01  sin(0.5 i) + 0.002 ((i%5)−2)]
```

Verified reference numbers (from drills):

Metric	Value	Source
Sensitivity vector `s` (rate, spread, vol, fx)	**(−0.51, −0.213, +5.55, +20.62)** approx	`var_methods_drill.sensitivities()`
P&L std (= √(sᵀΣs))	**2.5327**	drill 13
Historical VaR(99.5%)	**4.6781**	drill 13
Parametric VaR(99.5% / 99%)	**6.5238 / 5.8919**	drill 13
MC VaR(99.5% / 99%) seed=42, n=10000	**6.3411 / 5.8760**	drill 13
Gammas (rate, spread, vol, fx)	(1.855e-4, 4.287e-5, −1.064, 412.92)	`pnl_attribution_drill.gammas`
Explain scenario (+15bp, +10bp, +2vol, 0fx): full / first / second / unexp	−3.024 / −3.054 / +0.023 / +0.008	drill 12
5y 4% bond (single-asset hist VaR drill): base / DV01 / VaR(99.5%)	92.0706 / 0.0425 / 0.7363	`hist_var_drill.py`

Quick-jump matrix

Method	§	Pattern	Distribution assumption	Reval style
Historical VaR	1	empirical	none (data IS distribution)	FULL reval per scenario
Parametric VaR	2	analytical	factors ~ MV-normal, book LINEAR	sensitivity-based
Monte Carlo VaR	3	simulated	factors ~ assumed (e.g. MV-normal)	FULL reval per path
Expected Shortfall	4	mean of tail	inherits from method	inherits
Filtered Historical	5	empirical, rescaled	GARCH on factors	FULL reval
Cornish-Fisher	6	analytical	normal + skew/kurt adj	sensitivity-based
Marginal/Component VaR	7	analytical	normal	gradient of σ
Stressed VaR	8	historical, fixed window	"12-month stress period"	FULL reval
Kupiec / Christoffersen	9–10	LR test on hit-sequence	—	—
Basel traffic light	11	hit-count over 250 days	—	—

Part 0 — The Master Recipe

Every risk number in this document is a special case of **one identity**:

```
risk_number  =  T( distribution( P&L ) )
```

where `T` is a tail functional — `T = quantile` gives **VaR**, `T = mean | loss > VaR` gives **ES**, `T = sum of | hits|/N` gives a **backtest**.

The three things you ever change are: 1. **How you generate the P&L distribution** (history / parametric / MC). 2. **How you revalue** the book (sensitivity-based vs full reval). 3. **What functional T** you compute (quantile, mean-of-tail, gradient, scenario sum).

Everything else is bookkeeping.

The 4-step risk recipe

```
STEP 1.  IDENTIFY FACTORS.
         What random variables drive your P&L? (yield curve, credit spread,
         vol, FX, equity, commodity). Bucket where appropriate (KRD per tenor).

STEP 2.  GENERATE SHOCKS.
         Three choices:
            (a) HISTORICAL — replay actual daily factor moves (last 250d).
            (b) PARAMETRIC — assume MV-normal with covariance Σ; quantile = z·σ.
            (c) SIMULATED  — Cholesky Σ → draw Gaussian (or other) paths.

STEP 3.  REVALUE.
         Two choices:
            (i)  SENSITIVITY-BASED  — P&L ≈ s · shock  (Taylor, fast, linear).
            (ii) FULL REVALUATION   — P&L = V(shocked) − V(base)  (slow, exact).

STEP 4.  AGGREGATE / EXTRACT.
         T(distribution): quantile for VaR, mean-of-tail for ES, gradient for
         marginal VaR, sum for scenario explain.
```

That's it. The whole field of market-risk methodology is which option you pick in steps 2 and 3.

The substitution table

Method	Step 2 (shocks)	Step 3 (reval)	Step 4 (T)	Trade-off
Historical VaR	actual factor history `F`	FULL reval per row	quantile	no distribution assumption; bounded by window
Parametric VaR	implied: `σ_P&L = √(sᵀΣs)`	sensitivity (linear)	`z(α)·σ`	fast; assumes normal + linear; misses fat tails + gamma
Monte Carlo VaR	`L·z`, `L = chol(Σ)`, `z ~ N(0,I)`	FULL reval per path	quantile	captures non-linearity; model-dependent; heavy
Expected Shortfall	inherits	inherits	mean of P&L ≤ −VaR	coherent (subadditive); FRTB standard
Filtered Historical	`F` rescaled by GARCH σ ratio	FULL reval	quantile	history + vol updating; handles vol clustering
Cornish-Fisher	parametric + sample (skew, kurt)	sensitivity (linear)	adjusted quantile	quick fat-tail fix without MC

Method	Step 2 (shocks)	Step 3 (reval)	Step 4 (T)	Trade-off
Marginal/ Component	parametric	sensitivity	gradient of σ_P&L	risk decomposition by position
Stressed VaR	history from a fixed 12M stress window	FULL reval	quantile	locked-in stress; doesn't drift with calm markets

Three worked walk-throughs of the recipe

Walk-through 1 — Historical VaR

```
STEP 1.  Factors: 4 (rate, spread, vol, fx). History F: 250 × 4.

STEP 2.  Shocks = each row of F (replay actual daily moves).

STEP 3.  For each row i: V_i = pf.value(scenario(*F[i]).apply(base)); P&L_i = V_i − V₀.

STEP 4.  Sort P&L ascending; VaR(α) = −P&L[floor((1−α)·N)].
         At α=0.995, N=250: index 1 (second worst) → VaR = 4.6781.
```

That IS the Historical VaR formula in §1.

Walk-through 2 — Parametric VaR

```
STEP 1.  Same 4 factors.

STEP 2.  Σ = cov(F)  (4×4).  No path generation.

STEP 3.  Sensitivity vector s = (∂V/∂rate_bp, ∂V/∂spread_bp, ∂V/∂vol/0.01, ∂V/∂fx/0.01).
         Taylor: ΔV ≈ s · Δfactor.
         Distribution of ΔV: zero-mean Normal with std σ = √(sᵀΣs) = 2.5327.

STEP 4.  VaR(α) = z(α) · σ.   At α=0.995: 2.576 · 2.5327 = 6.5238.
```

Walk-through 3 — Monte Carlo VaR

```
STEP 1.  Same factors.

STEP 2.  L = chol(Σ);  draws ~ N(0, I)_{n × 4};  shocks = draws @ L.T  ~ N(0, Σ).

STEP 3.  For each path k: P&L_k = V(scenario(*shocks[k]).apply(base)) − V₀
         (FULL revaluation — captures gamma).

STEP 4.  VaR(α) = −percentile(P&L, (1−α)·100).
         At α=0.995, seed=42, n=10000: 6.3411.
```

Three recipes, three numbers, **one pattern**. The 6.34 (MC) vs 6.52 (param) gap is the **non-linearity correction** (full reval picks up convexity the linear Taylor misses). The 4.68 (hist) being LOWER than both is the **bounded-history correction** (the actual 250-day window's tail is thinner than a normal calibrated on the same covariance — the data simply hasn't seen as bad a day).

When the recipe breaks (flag and look up)

Situation	Why simple recipe fails	What to add
Heavy tails (fat-tailed factors)	normal underestimates extreme losses	Cornish-Fisher / EVT / t-copula MC
Vol clustering (GARCH effects)	recent vol > historical vol	Filtered Historical Sim (rescale by GARCH σ)
Non-linear book (lots of options)	linear approx misses gamma, vega-gamma	Full-reval MC (drop parametric)

Situation	Why simple recipe fails	What to add
Path-dependent products (barriers, MBS)	single shock can't reveal path	path-dependent MC
Wrong-way risk (CCR)	exposure ↑ when counterparty quality ↓	CVA with explicit correlation
Liquidity-adjusted	liquidation can't happen in 1 day	LVaR (add liquidation cost / horizon scaling)

The 3 patterns to recognise

```
P1  SENSITIVITY-BASED:   risk ≈ s · shock  (or √(sᵀΣs) for portfolios)
    Used by: parametric VaR, marginal VaR, Cornish-Fisher, FRTB SA.
    Fast O(#factors); falls down on non-linearity.

P2  FULL-REVALUATION:    risk = T( V(shocked) − V(base) over many shocks )
    Used by: historical VaR, MC VaR, stress P&L, ES (full-reval flavour).
    Exact; cost = O(#paths × pricer_cost).

P3  EMPIRICAL DISTRIBUTION:  risk = T(empirical_pnl_vector)
    Used by: historical VaR, FHS, backtests.
    No distribution assumption; bounded by window length.
```

Every method below is a combination. **Parametric** = P1 + analytical Normal. **Historical** = P2 + P3. **MC** = P2 + simulated Normal/copula. **Filtered Historical** = P2 + P3 + GARCH rescale.

Part I — VaR & Expected Shortfall

1. Historical VaR

> **TL;DR:** Rank-order actual daily P&Ls; report the (1−α)-quantile loss. No distribution assumption; bounded by window.

What: rank-order the **actual** historical daily P&Ls and report the loss at the (1−α) quantile. No distributional assumption — the data IS the distribution.

How it's calculated:

```
1. Take last N days of risk-factor changes:  F[i] for i = 0..N−1
2. For each day i:  shock today's market by F[i], revalue → P&L_i = V_i − V₀
3. Sort P&L ascending
4. VaR(α) = −P&L[ floor((1−α) · N) ]
```

Formula:

$$\text{VaR}(\alpha) = -\text{sorted}(P\&L)[\lfloor (1-\alpha)\cdot N \rfloor]$$

Input	Meaning
`N`	history window length (typically 250 trading days = 1 year)
`α`	confidence (0.99, 0.995)
`P&L_i`	full-reval P&L on scenario i = `V(F[i] applied) − V₀`

Toy example (single-asset, `hist_var_drill.py`):

```
5y 4% bond, notional 100, spread 150bp
N = 250 days; factor shocks d_rate_bp[i], d_spread_bp[i]

base price                = 92.0706
P&L_i = bond_price(rate_bump=d_rate_bp[i], spread_bp=150+d_spread_bp[i]) − 92.0706
min / max(P&L)            = −0.8011 / +0.8504

sorted P&L: index 0 = worst loss (P&L most negative)
VaR(99.5%) = −sorted[1]   = 0.7363
VaR(99.0%) = −sorted[2]   = 0.7033
```

Canonical Python:

```python
import numpy as np

def historical_var(pnl: np.ndarray, conf: float = 0.99) -> float:
    """Historical VaR via percentile. pnl is a vector of N P&Ls (loss = negative)."""
    return -np.percentile(pnl, (1 - conf) * 100)

# Equivalent index-based form (matches Solvency II SCR calibration):
def historical_var_idx(pnl: np.ndarray, conf: float = 0.995) -> float:
    s = np.sort(pnl)                      # ascending: most negative first
    idx = int(np.floor((1 - conf) * len(pnl)))
    return -s[idx]
```

Intuition / pattern: **P3 — empirical**. Bounded by window: anything not in your 250 days literally cannot show up. A book that hasn't seen a 2008 will print a tiny VaR until the day after one happens.

Limits: - Static window → vol clustering missed (today's calm hides yesterday's storm and vice versa). - 250 days at α=0.995 means the answer is **literally the 2nd-worst day** → enormous sampling error in the

tail; rolling-window VaR jumps when a single bad day enters/leaves. - Fix: **Filtered Historical** (§5) rescales each historical shock by the ratio `σ_today / σ_then`.

---

2. Parametric (delta-normal) VaR

> **TL;DR:** `VaR = z(α)·√(sᵀΣs)`. Closed-form, assumes joint-normal factors AND linear book.

What: assume factor changes are joint-normal with covariance Σ, and the book is **linear** in factors (sensitivities are constant). Then P&L is normal with std `σ = √(sᵀΣs)` and the quantile is closed-form.

How it's calculated:

```
1. Compute sensitivity vector s = (∂V/∂f_i)_i      (one bump per factor)
2. Estimate factor covariance Σ = cov(F)
3. P&L std:          σ = √( sᵀ Σ s )
4. VaR(α) = z(α) · σ      where z = norm.ppf
```

Formula:

$$\sigma^2_{P\&L} = s^\top \Sigma s$$
$$\mathrm{VaR}(\alpha) = z(\alpha) \cdot \sigma_{P\&L}$$

Input	Meaning
`s`	row vector of first-order sensitivities, one entry per factor
`Σ`	factor covariance matrix; same units as `s`'s shock unit
`z(α)`	std-normal quantile — `norm.ppf(α)`; 99% → 2.326, 99.5% → 2.576

Toy example (`var_methods_drill.py`):

```
s = [ ∂V/∂rate_bp, ∂V/∂spread_bp, ∂V/∂vol/0.01, ∂V/∂fx/0.01 ]
Σ = cov(F)   (4×4; F = 250×4 factor history)

σ_P&L      = √(sᵀ Σ s)    = 2.5327
VaR(99.5%) = 2.576 · 2.5327 = 6.5238
VaR(99.0%) = 2.326 · 2.5327 = 5.8919
```

Canonical Python:

```python
import numpy as np
from scipy.stats import norm

def parametric_var(s: np.ndarray, cov: np.ndarray, conf: float = 0.99) -> float:
    """Delta-normal VaR. s: sensitivity vector. cov: factor covariance (same shock units)."""
    pnl_var = float(s @ cov @ s)
    pnl_std = np.sqrt(pnl_var)
    return norm.ppf(conf) * pnl_std

# Toy: 1 rate factor (DV01=100 per bp), 1 FX factor (FX-Δ=50 per 1.0)
s   = np.array([100.0, 50.0])
cov = np.diag([(8.0)**2, (0.005)**2])    # bp² and unit² — factor units MUST match s
print(parametric_var(s, cov, 0.99))      # → 1862.5 ($)
```

Intuition / pattern: **P1 — sensitivity-based, analytical Normal**. Closed-form, scales O(#factors²) regardless of book size. The cost is a **double assumption**: factors normal AND book linear. Two errors that can cancel or compound:

- Skew/heavy tails → Normal underestimates → VaR too LOW.
- Positive gamma (long options) → linear underestimates payoff → VaR too HIGH.
- Negative gamma (short options) → linear overestimates payoff → VaR too LOW.

When it works: a vanilla rates/credit book with no concentrated optionality. When in doubt, do MC + compare.

---

3. Monte Carlo VaR (full revaluation)

> **TL;DR:** Cholesky-simulate factor paths; full-revalue book per path; take percentile. Only method that handles non-linearity + arbitrary distribution.

What: simulate factor paths under an assumed joint distribution (typically MV-normal via Cholesky), then full-revalue the book on each path and take the empirical quantile.

How it's calculated:

```
1. Σ = factor covariance
2. L = cholesky(Σ)                (lower-triangular,  L Lᵀ = Σ)
3. Draw  z ~ N(0, I)_{n × #factors}
4. shocks = z @ Lᵀ                (~ N(0, Σ) jointly normal)
5. For each path k:  P&L_k = V(scenario(*shocks[k]).apply(base)) − V₀     (FULL reval)
6. VaR(α) = −percentile(P&L, (1−α)·100)
```

Formula:

```
                  L Lᵀ = Σ;   shocks = z·Lᵀ ~ N(0, Σ)
                  P&L_k = V_full_reval(base + shocks_k) − V₀
                  VaR(α) = −quantile(P&L, 1−α)
```

Input	Meaning
`n`	number of MC paths (10,000 typical; 100,000 for tail accuracy)
`seed`	RNG seed — **always set explicitly** for reproducible risk numbers
`Σ`	factor covariance (units must match the pricer's shock interface)

Toy example (`var_methods_drill.py`):

```
Same s, Σ as parametric.
n_paths = 10_000, seed = 42
VaR(99.5%) = 6.3411    ← LOWER than parametric 6.5238 (gamma cushions losses)
VaR(99.0%) = 5.8760    ← almost identical to parametric 5.8919 at 99%
```

The 99% match shows the book is mostly linear; the 99.5% gap reveals the tail non-linearity that parametric ignores.

Canonical Python:

```python
import numpy as np

def monte_carlo_var(value_fn, base, cov: np.ndarray,
                    conf: float = 0.99, n_paths: int = 10_000, seed: int = 42) -> float:
    """Cholesky MC VaR with FULL revaluation. value_fn(base, shocks_row) -> V.
    cov shape: (#factors, #factors); units must match value_fn's expected shock vector.
    """
    L   = np.linalg.cholesky(cov)
    rng = np.random.default_rng(seed)
    z   = rng.standard_normal((n_paths, cov.shape[0]))
    shocks = z @ L.T                                  # ~ N(0, Σ)
    v0 = value_fn(base, np.zeros(cov.shape[0]))
    pnl = np.array([value_fn(base, s) - v0 for s in shocks])
    return float(-np.percentile(pnl, (1 - conf) * 100))
```

Intuition / pattern: **P2 + simulated distribution**. The only method that combines **(a) distributional flexibility** (swap N(0,Σ) for t-copula, NIG, GARCH-Cholesky, …) and **(b) full revaluation** (captures

gamma, vega-gamma, cross-gamma). Cost is `n_paths × pricer_cost` — for slow pricers (MBS, Bermudan), this can run for hours.

Variance reduction: antithetic variates, control variates, importance sampling, quasi-Monte Carlo (Sobol). Cuts standard error of the quantile estimator.

---

4. Expected Shortfall (CVaR)

> **TL;DR:** Average loss IN the tail, not at the tail boundary. Coherent (subadditive); FRTB standard at α=0.975.

What: the **average loss given you are in the tail**. ES_α = E[L | L ≥ VaR_α]. Replaces VaR under FRTB IMA because it is **coherent** (subadditive — diversifying always helps).

How it's calculated:

```
1. Generate the P&L distribution (any of §1–3).
2. Find VaR_α = −quantile(P&L, 1−α).
3. ES_α = − mean( P&L | P&L ≤ −VaR_α )
        = − mean( P&Ls in the worst (1−α) fraction )
```

Formula:

```
                      ES_α  =  E[ −P&L  |  P&L ≤ −VaR_α ]

                      Empirical:  ES_α = − mean(sorted_pnl[:k])
                                          k = ⌈(1−α)·N⌉
```

Parametric (Normal) closed-form:

```
ES_α = σ · φ(z(α)) / (1 − α)            (zero-mean P&L)
At α = 0.975: ES = σ · 2.338             (FRTB calibration)
At α = 0.99:  ES = σ · 2.665
```

Toy example (same drill 13 portfolio, parametric Normal):

```
σ = 2.5327
ES(97.5%) parametric = 2.5327 · φ(1.96)/0.025 = 2.5327 · 2.3378 = 5.9220
ES(99%)   parametric = 2.5327 · φ(2.326)/0.01 = 2.5327 · 2.6652 = 6.7501

(Compare VaR(99%) = 5.89; ES(97.5%) is FRTB's "10-day liquidity-horizon-scaled" replacement.)
```

Canonical Python:

```python
import numpy as np
from scipy.stats import norm

def historical_es(pnl: np.ndarray, conf: float = 0.975) -> float:
    """Empirical ES: average loss in the worst (1−conf) fraction."""
    s = np.sort(pnl)
    k = max(1, int(np.ceil((1 - conf) * len(pnl))))
    return float(-s[:k].mean())

def parametric_es(sigma: float, conf: float = 0.975) -> float:
    """Normal-distribution ES (zero-mean P&L), closed-form."""
    z = norm.ppf(conf)
    return float(sigma * norm.pdf(z) / (1 - conf))
```

Intuition / pattern: ES sums the WHOLE tail, not just the boundary. **Coherent**: ES(A+B) ≤ ES(A) + ES(B) always (VaR can violate this for non-elliptical distributions, especially long-tail credit). FRTB IMA uses ES(97.5%) on a stressed window with liquidity-horizon scaling per factor class.

Key relation (Normal):

```
ES_α(Normal) / VaR_α(Normal) ≈ φ(z(α)) / [(1−α)·z(α)]
At α=0.99:  ratio ≈ 1.145    →  ES is ~14.5% bigger than VaR for normal P&L
At α=0.975: ratio ≈ 1.193
```

---

5. Filtered Historical Simulation (FHS)

> **TL;DR:** Historical VaR with each old shock rescaled by today's GARCH vol. Solves stale-vol problem in calm/crisis transitions.

What: historical simulation, but each old shock is **rescaled** by the ratio of today's volatility to the volatility on the day the shock happened. Combines history's empirical distribution with GARCH's vol updating.

How it's calculated:

```
1. Estimate a GARCH(1,1) on each factor: σ̂t = √(ω + α·r²_{t−1} + β·σ²_{t−1})
2. Standardise historical innovations:    z_i = F_i / σ̂i        (per factor)
3. Rescale by today's vol:                shock_i = σ̂today · z_i
4. Apply each rescaled shock, full-reval, VaR as in §1.
```

Formula:

```
                    shock_i_today = (σ̂today / σ̂i_then) · F_i
                    then proceed as historical VaR
```

Toy example:

```
History contains a calm period (σ̂= 0.5%) and the current state is vol-stressed (σ̂= 2%).
A historical move of 10bp on a calm day → rescaled to (2.0/0.5) · 10 = 40bp today.
Tail VaR rises commensurately even though the "shock library" is the same 250 days.
```

Canonical Python (sketch — full GARCH fit usually via `arch` library):

```python
import numpy as np
from arch import arch_model                  # uv add arch

def fhs_shocks(returns_history: np.ndarray) -> np.ndarray:
    """Rescale a 1-factor history by today's GARCH(1,1) vol."""
    am = arch_model(returns_history * 100, mean="Zero", vol="GARCH", p=1, q=1)
    res = am.fit(disp="off")
    sigma_t = res.conditional_volatility / 100        # per-day estimated σ
    z = returns_history / sigma_t                     # standardised innovations
    sigma_today = sigma_t[-1]
    return sigma_today * z                            # rescaled shock library
```

Intuition / pattern: **P2 + P3 + GARCH rescale**. Solves historical's "stale during crises" problem without giving up the empirical distribution. Standard for trading-desk VaR on liquid factors (FX, rates).

Limits: if the GARCH model is mis-specified (regime shift, structural break), the rescale is wrong. Always plot `σ̂t` against realised vol as a sanity check.

5b. FHS — bootstrap variant (production form)

> **TL;DR:** Draw standardised residuals with replacement (instead of rescaling each day). What most production desks actually run.

What: most production desks don't simply *rescale* each historical day. They **bootstrap** standardised residuals — draw `z*` with replacement from the empirical innovation distribution, then scale by today's

GARCH σ. Captures both **vol updating** (from GARCH) and **distributional shape** (from history, including its fat tails).

How it's calculated:

```
1. Fit GARCH(1,1) on each factor; extract standardised residuals z_i = r_i / σ̂_i.
2. For each of n_paths simulated days:
     draw  z* with replacement from {z_1, …, z_T}      # empirical bootstrap
     shock_path = σ̂_today · z*
3. Apply each shock, full-reval → P&L distribution.
4. VaR(α) = −percentile(P&L, (1−α)·100)  as usual.
```

Formula:

```
                    z* ~ Empirical({r_i / σ̂_i})  (with replacement)
                    shock = σ̂_today · z*
                    Then standard FULL-reval Historical VaR pipeline.
```

Toy example:

```
History 250 days, GARCH σ̂_today = 1.5%.   Standardised residuals z = r/σ̂_have:
   empirical std = 1.0 (by construction)
   empirical skew = −0.4 (negative-tail fatter)
   empirical kurt = 5.2 (excess; Normal would be 0)

n_paths = 10_000:  draw 10k z* values with replacement; shock = 1.5% · z*.
Resulting tail at 99% is fatter than parametric σ·z(0.99) because the BOOTSTRAP
preserves the empirical kurt, not just the variance.
```

Canonical Python:

```
import numpy as np

def fhs_bootstrap_shocks(returns_history: np.ndarray, sigma_today: float,
                         n_paths: int = 10_000, seed: int = 42) -> np.ndarray:
    """FHS via bootstrap of standardised residuals.
    Requires a fitted σ̂_t history (use GARCH or EWMA — passed in implicitly via
    standardised residuals z_i = r_i / σ̂_i)."""
    rng = np.random.default_rng(seed)
    # Assume returns_history is already standardised by σ̂_t per-day:
    z_star = rng.choice(returns_history, size=n_paths, replace=True)
    return sigma_today * z_star
```

Intuition / pattern: **GARCH for the SCALE, empirical for the SHAPE**. Solves both: (a) regime-aware vol from GARCH, (b) fat tails / skew from history — without the parametric assumption fight. Standard for FX, rates, equity-index VaR engines at most major banks.

Variant: **Christoffersen-Diebold filter** instead of GARCH (kernel-smoothed) — same idea, smoother σ̂_t.

---

6. Cornish-Fisher VaR

> **TL;DR:** Adjust Normal quantile by sample skew + excess kurt. Quick fat-tail fix without MC; fails for very heavy tails.

What: a fat-tail correction to parametric VaR that adjusts the Gaussian quantile by the **sample skew and kurtosis**. One line of arithmetic; no MC.

How it's calculated:

```
1. Compute sample mean μ, std σ, skew S, excess kurtosis K of returns.
2. Cornish-Fisher quantile:
       z_CF(α) = z + (z² − 1)·S/6
                   + (z³ − 3z)·K/24
                   − (2z³ − 5z)·S²/36
3. VaR(α) = −(μ + z_CF · σ)            (z is the standard-Normal quantile of 1−α; negative for tails)
```

Formula (the corrected quantile):

```
                  z_CF(α) = z + (z²−1)·S/6 + (z³−3z)·K/24
                                           − (2z³−5z)·S²/36
                  VaR(α)  = −(μ + z_CF · σ)
```

Input	Meaning
`z`	std-Normal quantile of `1−α` (negative for tails — e.g. −2.326 for 99%)
`S`	sample skew of returns (negative S = downside-heavy tail)
`K`	sample EXCESS kurtosis (= kurt − 3; positive for fat tails)

Toy example:

```
Sample equity returns: μ=0, σ=0.012, skew S=−0.5, excess kurt K=4.
At α=0.99:  z = −2.326.
z_CF = −2.326 + (5.41−1)(−0.5)/6 + (−12.58+6.98)(4)/24 − (−25.16+11.63)(0.25)/36
     = −2.326 − 0.367  − 0.933  + 0.094
     ≈ −3.532
VaR_99 = 0.012 · 3.532 = 4.24%   (vs Normal VaR_99 = 0.012 · 2.326 = 2.79%)
```

The fat-tail correction here makes VaR ~50% larger than the Gaussian assumption.

Canonical Python:

```
import numpy as np
from scipy.stats import norm, skew, kurtosis

def cornish_fisher_var(returns: np.ndarray, conf: float = 0.99) -> float:
    mu, sd = returns.mean(), returns.std(ddof=1)
    S = skew(returns)
    K = kurtosis(returns)                              # already excess (Fisher def)
    z = norm.ppf(1 - conf)                             # negative for tail
    z_cf = z + (z**2 - 1)*S/6 + (z**3 - 3*z)*K/24 - (2*z**3 - 5*z)*S**2/36
    return float(-(mu + z_cf * sd))
```

Intuition / pattern: **P1 + parametric with moment corrections**. Cheap fat-tail fix when MC is too expensive but Normal is too thin. Falls apart for very heavy tails (|S|>2 or K>10) — at that point use EVT or t-copula MC.

---

7. Marginal / Component / Incremental VaR

> **TL;DR:** MVaR = per-$ trade sizing; CVaR = limit attribution (Σ = total VaR via Euler); IVaR = full-removal what-if.

What: decomposes total VaR by position. Three flavours, three different questions:

- **Marginal VaR (MVaR_i)**: per-$ sensitivity — `∂VaR / ∂w_i`. "What's the risk per extra dollar in position i?"
- **Component VaR (CVaR_i)**: contribution to total — `w_i · MVaR_i`. **Sums to total VaR**.
- **Incremental VaR (IVaR_i)**: full removal — `VaR(portfolio) − VaR(portfolio without position i)`. The actual "what if I close this position".

How it's calculated (under MV-normal):

```
σ²_P&L = wᵀ Σ w                              (w = position vector, Σ = position cov)
MVaR_i = z(α) · (Σ w)_i / σ_P&L              (∂σ/∂w_i = (Σw)_i / σ; ∂VaR = z · ∂σ)
CVaR_i = w_i · MVaR_i                         (sums to VaR by Euler's theorem)
IVaR_i = VaR(w) − VaR(w_{−i})                 (re-run VaR with w_i = 0)
```

Formula:

```
                    MVaR_i = z(α) · (Σ w)_i / √(wᵀ Σ w)
                    CVaR_i = w_i · MVaR_i
                    Σ CVaR_i = VaR_total          ✓ Euler

   Σ here is the POSITION-return covariance (N positions × N positions),
   NOT the FACTOR covariance used in §2. The two are related by Σ_pos =
   B Σ_factor Bᵀ where B is the per-position sensitivity matrix.

   Euler closure: σ = √(wᵀΣw) is homogeneous-of-degree-1 in w, so
   Σ w_i · ∂σ/∂w_i = σ  → Σ CVaR_i = z·σ = VaR_total exactly.
```

Toy example:

```
2 positions:  w = [60, 40].  σ_1 = 0.02, σ_2 = 0.01, ρ = 0.5.
Σ = [[4e-4, 1e-4], [1e-4, 1e-4]]
σ_P&L = √(60·60·4e-4 + 2·60·40·1e-4 + 40·40·1e-4) = √(1.44 + 0.48 + 0.16) = √2.08 = 1.4422

(Σ w) = [60·4e-4 + 40·1e-4, 60·1e-4 + 40·1e-4] = [0.028, 0.010]
MVaR_1 = 2.326 · 0.028 / 1.4422 = 0.04514
MVaR_2 = 2.326 · 0.010 / 1.4422 = 0.01613
CVaR_1 = 60 · 0.04514 = 2.708    ← 81% of total
CVaR_2 = 40 · 0.01613 = 0.645    ← 19% of total
Σ CVaR = 3.354    ✓  = z(α)·σ_P&L = 2.326 · 1.4422
```

Canonical Python:

```python
import numpy as np
from scipy.stats import norm

def marginal_var(w: np.ndarray, cov: np.ndarray, conf: float = 0.99) -> np.ndarray:
    """MVaR per position. cov: position-level (NOT factor-level) covariance."""
    sigma = np.sqrt(w @ cov @ w)
    return norm.ppf(conf) * (cov @ w) / sigma

def component_var(w: np.ndarray, cov: np.ndarray, conf: float = 0.99) -> np.ndarray:
    return w * marginal_var(w, cov, conf)
```

Intuition / pattern: **P1 + Euler decomposition**. Component VaR is the **risk attribution** of choice for limits: tells you which desks/positions are eating the VaR budget. Marginal VaR is the **trade-sizing** number: a position's MVaR > its expected return / σ implies the position is destroying risk-adjusted return.

IVaR is what you compute before a trade — "if I add this $10M of position X, how does VaR change?" — and it can differ materially from `w_X · MVaR_X` because adding a position changes the whole correlation structure.

7b. Risk Budgeting / Equal Risk Contribution (ERC)

> **TL;DR:** Inverse of §7 — solve for weights producing target contributions. ERC = equal risk per position. Maillard iteration.

What: the **inverse** of §7. Instead of measuring contributions of a given book, **solve for weights** that produce a target contribution profile. Equal Risk Contribution (ERC) is the special case where every position contributes the same risk.

How it's calculated:

```
Decision variable:  w
Constraint:         w_i · (Σw)_i / √(wᵀΣw)  =  b_i · √(wᵀΣw)  ∀ i
                    Σ w_i = 1,  w_i ≥ 0

   b_i = target risk-budget weight (Σ b_i = 1).
   ERC: b_i = 1/N for all i.

Solve via convex optimisation; ERC has a known iterative solution (Maillard 2010):
   w_i^(k+1) = (1 / (Σw)_i^(k)) ; then normalise.
```

Formula:

```
                    Position i's risk contribution share:
                        rc_i = w_i · (Σw)_i / (wᵀΣw)
                    Risk budget:  rc_i = b_i  ∀ i
                    ERC special case:  rc_i = 1/N  ∀ i
```

Toy example (3 assets, σ=(20%, 10%, 15%), ρ=0.3 between all pairs):

```
ERC weights ≈ (0.221, 0.476, 0.303)  — inversely related to vol, but adjusted
                                       for correlation.
Each position contributes 1/3 of total portfolio risk.
Compare to equal-weight (1/3 each): more vol budget on the high-vol asset.
```

Canonical Python (Maillard iterative; cvxpy alternative shown commented):

```python
import numpy as np

def erc_weights(cov: np.ndarray, max_iter: int = 1000, tol: float = 1e-9) -> np.ndarray:
    """Equal Risk Contribution weights via Maillard fixed-point iteration."""
    n = cov.shape[0]
    w = np.ones(n) / n
    for _ in range(max_iter):
        marginal = cov @ w
        rc = w * marginal
        target = rc.sum() / n            # target contribution = total / N
        w_new = w * (target / rc)
        w_new /= w_new.sum()
        if np.max(np.abs(w_new - w)) < tol:
            return w_new
        w = w_new
    return w

# General risk-budgeting via convex opt (uv add cvxpy):
# import cvxpy as cp
# w = cp.Variable(n, nonneg=True)
# constraints = [cp.sum(w) == 1]
# # Convex reformulation of rc_i = b_i via auxiliary variable; see Spinu (2013).
```

Intuition / pattern: **same Euler decomposition as §7 — used as a constraint instead of a report**. Two huge buy-side applications: - **ERC SAA**: equal risk-contribution portfolio is a robust starting allocation when expected returns are uncertain (Maillard, Roncalli, Teiletche 2010). - **Risk-parity funds** (Bridgewater All-Weather, AQR Risk Parity) scale leverage so each asset class contributes equally to portfolio vol — outperforms equal-weight on Sharpe through better tail behaviour, though leverages bonds heavily.

Pitfall: ERC requires **invertible Σ**. For N ≈ T use Ledoit-Wolf (§28) first. ERC also tends to **overweight low-vol assets** — combine with a leverage constraint or a vol target if running unconstrained.

---

8. Stressed VaR (sVaR)

> **TL;DR:** Historical VaR on a FIXED 12-month stress window (e.g. 2008). Basel II.5: capital = max(VaR, sVaR)·k.

What: Historical VaR run on a **fixed 12-month window of significant financial stress** instead of the rolling-recent window. Basel II.5 addition (2009) and FRTB IMA component.

How it's calculated:

```
1. Pick a stress window — typically the 12 months that produced the largest losses
   for the current book (e.g. mid-2008, March 2020).
2. Run historical VaR (or ES) on that fixed window. Window does NOT roll.
3. Capital = max( recent_VaR, sVaR ) · multiplier  (Basel II.5).
```

Formula:

$$\text{sVaR}(\alpha) = \text{Historical\_VaR}(\alpha) \text{ on FIXED stress window}$$
$$\text{Capital}_{\text{market}} = \max(\text{VaR}_{60d,\text{avg}}, \text{sVaR}_{60d,\text{avg}}) \cdot m$$

Toy example:

```
Current 250-day window σ = 1% (calm year).  Historical VaR(99%) = 2.3.
Stress window (Sep 2008 – Sep 2009) σ = 4%.  sVaR(99%) = 9.2.
Basel II.5 capital ≈ (2.3 + 9.2) · 3 = 34.5    (vs old VaR-only = 6.9 – almost 5× higher)
```

Canonical Python:

```python
def stressed_var(factor_history_full: np.ndarray, stress_start: int,
                 stress_end: int, pf_value_fn, base, conf: float = 0.99) -> float:
    """Historical VaR over a fixed [stress_start:stress_end] slice."""
    F = factor_history_full[stress_start:stress_end]
    v0 = pf_value_fn(base)
    pnl = np.array([pf_value_fn(scenario(*f).apply(base)) - v0 for f in F])
    return float(-np.percentile(pnl, (1 - conf) * 100))
```

Intuition / pattern: **P2 + P3 with locked window**. Fixes Historical VaR's "stale during calm markets" problem by forcing the capital number to remember 2008. Each bank picks its own stress window subject to regulator approval; the choice must be the one that produces the **highest** sVaR for the current portfolio.

FRTB ES analogue: stressed ES on a reduced-set of risk factors (RRF) — the chosen window must be the worst for the **reduced** set.

Part II — Backtesting

The question every backtest answers: **does the realised hit rate match the VaR's claimed confidence level?**

A 99% VaR should be breached ~1% of days. In 250 days: 2.5 hits expected. 0 hits = model too conservative; 8 hits = model under-estimating tail. The tests below quantify how surprising the observed hit count is.

```
Hit indicator:   I_t = 1 if P&L_t < −VaR_t,  else 0
Hit sequence:    I = [I_1, I_2, …, I_T]
Observed hits:   x = Σ I_t
Expected hits:   T·(1−α)
```

9. Kupiec POF test

TL;DR: LR test on observed hit count vs 1−α expected. $\chi^2(1)$; reject if LR > 3.841. Catches under/over-estimation; ignores clustering.

What: Likelihood-ratio test that the **unconditional hit rate** equals the claimed `(1−α)`. Tests for over/under-estimation but ignores clustering.

How it's calculated:

```
Null H₀:  p = π,  where p = observed hit rate, π = 1−α (theoretical)
LR_POF = −2 · ln[ ((1−π)^(T−x) · π^x) / ((1−p)^(T−x) · p^x) ]
       ~ χ²(1)    under H₀
Reject H₀ at 5%:  LR_POF > 3.841
```

Formula:

```
                LR_POF = 2 [ x·ln(p/π) + (T−x)·ln((1−p)/(1−π)) ]

                    T  = backtest days,  x = hits,  p = x/T,  π = 1−α
                Reject if LR_POF > χ²₁,₀.₀₅ = 3.841
```

Input	Meaning
`T`	backtest window (250 days standard)
`x`	observed number of VaR breaches
`π = 1−α`	theoretical hit probability (0.01 at 99%)

Toy example:

```
T = 250, α = 99% so π = 0.01, expected hits = 2.5.
Observed x = 8 hits  →  p = 0.032

LR_POF = 2 [ 8·ln(0.032/0.01) + 242·ln(0.968/0.99) ]
       = 2 [ 8·1.163 + 242·(−0.0225) ]
       = 2 [ 9.30 − 5.45 ]
       = 7.71            >  3.841   →  REJECT (model under-estimates tail)
```

Canonical Python:

```
import numpy as np
from scipy.stats import chi2

def kupiec_pof(hits: np.ndarray, conf: float = 0.99) -> tuple[float, float, bool]:
    """Kupiec POF test. Returns (LR statistic, p-value, reject_at_5%)."""
    T = len(hits)
    x = int(hits.sum())
    pi = 1 - conf
    p = x / T
    if x == 0:
        lr = -2 * T * np.log(1 - pi)
    elif x == T:
        lr = -2 * T * np.log(pi)
    else:
        lr = 2 * (x*np.log(p/pi) + (T-x)*np.log((1-p)/(1-pi)))
    pval = 1 - chi2.cdf(lr, df=1)
    return float(lr), float(pval), bool(lr > 3.841)
```

Intuition / pattern: counts hits, doesn't care WHEN. A model that produced exactly 2 hits both on consecutive days passes Kupiec but fails Christoffersen (§10).

---

10. Christoffersen independence test

> **TL;DR:** LR on hit-sequence transitions. Catches vol-blind models (right hit count, all in one week). Joint LR_CC = POF + IND.

What: Likelihood-ratio test that **hits are independent** day-to-day. Catches the failure mode where a vol-blind VaR clusters all its breaches during a single bad week.

How it's calculated:

```
Build 2×2 transition table on the hit sequence:
     T_00 = # (I_{t−1}=0, I_t=0)        T_01 = # (I_{t−1}=0, I_t=1)
     T_10 = # (I_{t−1}=1, I_t=0)        T_11 = # (I_{t−1}=1, I_t=1)
π_01 = T_01/(T_00+T_01),  π_11 = T_11/(T_10+T_11),  π = (T_01+T_11)/(T_00+T_01+T_10+T_11)
LR_ind ~ χ²(1).

Christoffersen joint test: LR_CC = LR_POF + LR_ind ~ χ²(2)   (rejection threshold 5.991)
```

Formula:

```
                Compact form (using saturated vs constrained likelihoods):

                LR_ind = −2 · ln[
                   (1−π)^(T_00+T_10) · π^(T_01+T_11)
                 ──────────────────────────────────────────
                   (1−π_01)^T_00 · π_01^T_01 ·
                   (1−π_11)^T_10 · π_11^T_11
                ]

                Equivalent expanded form:
                LR_ind = 2 [ T_01·ln(π_01/π) + T_11·ln(π_11/π)
                           + T_00·ln((1−π_01)/(1−π))
                           + T_10·ln((1−π_11)/(1−π)) ]
                Reject independence if LR_ind > 3.841 (χ²₁)
                Reject joint if LR_CC = LR_POF + LR_ind > 5.991 (χ²₂)
```

Toy example:

```
T=250, observed hits: 6, with pattern [...000 1 1 1 1 1 1 000...]  (all 6 clustered).
T_00 = 243, T_01 = 1, T_10 = 1, T_11 = 5
π_01 = 1/244 = 0.0041,  π_11 = 5/6 = 0.833,  π = 6/250 = 0.024
LR_ind huge (≈ 50)  →  REJECT independence  →  model is vol-blind.
```

Canonical Python:

```
import numpy as np
from scipy.stats import chi2

def christoffersen_ind(hits: np.ndarray) -> tuple[float, float, bool]:
    """Independence test on hit sequence. Returns (LR, p-value, reject)."""
    T = len(hits)
    T00 = T01 = T10 = T11 = 0
    for i in range(1, T):
        if   hits[i-1]==0 and hits[i]==0: T00 += 1
        elif hits[i-1]==0 and hits[i]==1: T01 += 1
        elif hits[i-1]==1 and hits[i]==0: T10 += 1
        else: T11 += 1
    p01 = T01 / max(T00 + T01, 1)
    p11 = T11 / max(T10 + T11, 1)
    p   = (T01 + T11) / max(T00 + T01 + T10 + T11, 1)
    eps = 1e-12
    def safe_log(x): return np.log(max(x, eps))
    lr = 2 * (T01*safe_log(p01/p) + T11*safe_log(p11/p)
              + T00*safe_log((1-p01)/(1-p)) + T10*safe_log((1-p11)/(1-p)))
    return float(lr), float(1 - chi2.cdf(lr, df=1)), bool(lr > 3.841)
```

Intuition / pattern: **clustering ≠ frequency**. Kupiec passes if hit count is OK; Christoffersen fails if those hits all happened in a week. A model with stale vol passes Kupiec on a quiet year and fails Christoffersen the year a crisis hits.

---

11. Basel traffic-light test

> **TL;DR:** Hit count over 250d at 99% → green/yellow/red zone → capital multiplier k ∈ [3.0, 4.0]. Punitive asymmetric structure.

What: regulator's hit-count test on a 99% one-day VaR over 250 days. Trivially simple, materially expensive — wrong colour multiplies your capital.

How it's calculated:

Hits	Zone	Capital multiplier
0–4	**Green**	3.0
5–9	**Yellow**	3.4 → 4.0 (steps)
≥10	**Red**	4.0 + supervisory review

Hits	Multiplier (k)
5	3.40
6	3.50
7	3.65
8	3.75
9	3.85
≥10	4.00

Formula:

```
Market_Risk_Charge = max(VaR_t, avg_60d_VaR·k)

    k = traffic-light multiplier (above)
```

Toy example:

```
Bank's VaR avg over last 60 days = $50M. Backtest 250d shows 6 hits → yellow, k=3.50.
Capital charge ≈ max(today's VaR, 50 · 3.50) = max(today, 175) = $175M.

Same VaR with 10 hits → red, k=4.00 → $200M.    +14% capital penalty for 4 extra hits.
```

Canonical Python:

```
def basel_traffic_light(hits: int) -> tuple[str, float]:
    """Basel II/III VaR backtest. Input: hit count over 250d at 99%."""
    if hits <= 4:   return "GREEN", 3.0
    if hits <= 9:
        k = {5: 3.40, 6: 3.50, 7: 3.65, 8: 3.75, 9: 3.85}[hits]
        return "YELLOW", k
    return "RED", 4.0
```

Intuition / pattern: a **business decision metric**, not a statistical one. The expected hit count under a correct 99% VaR over 250 days is 2.5, with a 95% range of [0, 5]. Hit counts of 5+ are statistically plausible but regulatorily punished — the asymmetric threshold structure pushes banks toward conservative VaR.

---

12. ES backtesting (Acerbi-Szekely)

TL;DR: Z-statistic on hit-weighted realised loss vs forecast ES. Mandated by FRTB IMA; low power at α=0.975 (~6 hits / 250d).

What: traditional Kupiec/Christoffersen test only `P[L > VaR]`. **ES requires testing the conditional expectation of the tail**, which is harder because ES isn't elicitable (no scoring function whose minimiser is ES alone).

Acerbi & Szekely (2014) proposed three tests; **Test 2** ("unconditional coverage of ES") is the practical workhorse.

How it's calculated (Test 2):

```
Define hit-weighted residual:
    Z = (1/T) · Σ_t  [ X_t · I_t / (ES_t · (1−α)) ]     − 1

   where X_t = realised loss on day t (X_t = −P&L_t),
         I_t = 1 if hit (P&L_t < −VaR_t),
         ES_t = forecast ES at α.

Under H₀ ("model correct"):  E[Z] = 0.
Negative Z → model under-estimates ES (capital too low).
Sign / magnitude tested via Monte Carlo p-values (no closed-form).
```

Formula:

```
                  Z = (1/T) · Σ_t [ X_t · I_t / (ES_t · (1−α)) ] − 1
                  H₀ : E[Z] = 0   →   reject for Z < 0 (typically)
```

Toy example:

```
T = 250, α = 0.975, expected hits = 6.25.
Observed: 8 hits with avg-loss-on-hit = 3.2; model ES_t ≈ 2.8 every day.

Z = (1/250) · 8 · 3.2 / (2.8 · 0.025) − 1
  = 0.0256 · 45.71 − 1                ← wait, simpler form:
  = mean( X·I ) / (ES·(1−α)) − 1
  = (8/250)·3.2 / (2.8 · 0.025) − 1
  = 0.1024 / 0.0700 − 1
  = 0.4633     →   model UNDER-estimates ES by ~46%.
```

Canonical Python:

```
import numpy as np

def acerbi_szekely_test2(pnl: np.ndarray, var_forecast: np.ndarray,
                         es_forecast: np.ndarray, conf: float = 0.975) -> float:
    """Returns Z; negative-and-large is bad (model underestimates ES)."""
    loss = -pnl
    hits = (pnl < -var_forecast).astype(float)
    return float((loss * hits / (es_forecast * (1 - conf))).mean() - 1)
```

Intuition / pattern: ES is the **average loss given a hit**, so the test only "uses" the hits — sample size at α=0.975 is `~T·(1−α) = 6.25` hits per 250 days, giving tiny power. **Daily forecasts of ES must be archived** for the backtest to even be possible. FRTB IMA mandates ES at α=0.975 with this test on the trading portfolio's hypothetical P&L; failure triggers fallback to FRTB SA.

Part III — Risk Sensitivities

Two reasons to compute sensitivities: (a) drive **parametric VaR / capital** (FRTB SA is entirely sensitivity-based); (b) drive **trading-desk hedges** (target = zero net Greek).

A sensitivity is a **directional derivative**: `s_f = ∂V/∂f`. The implementation is almost always **bump-and-revalue** (finite difference), not analytical AD — for legacy reasons + because most banks' pricers are not AD-instrumented.

13. First-order sensitivities

TL;DR: `s_f = [V(base+δ_f) − V(base)] / δ_f` per factor. Bumps: 1bp (rates/spread), 0.01 (vol/FX). PICK ONE SIGN convention and label it.

The universal recipe:

```
s_f  =  [ V(base with f bumped by δ) − V(base) ]  /  δ
                                                     └─ shock UNIT (1bp, 1.0 vol, 1% FX)
                                                        chosen to make s_f naturally scaled
```

Sensitivity	Factor	Bump unit	Typical sign for a long bond
DV01 / PV01	parallel zero curve	+1 bp	negative (price ↓ when rate ↑)
CS01	credit spread	+1 bp	negative
IE01	breakeven inflation	+1 bp	positive for linker
FX-Δ	FX shock (% of spot)	+1% (0.01)	positive for long-foreign
Vega	implied vol shock	+1 vol pt (0.01)	positive for long-option

Convention pitfall: half the world reports DV01 = `V(rate) − V(rate+1bp)` (loss-given-up-move; positive for a long bond), the other half reports `∂V/∂r` (negative for a long bond). **Pick one and label it explicitly**. The `pnl_attribution_drill.py` convention is the latter: `s = V(bumped) − V(base)`, so DV01 of a long bond is negative.

Formula:

```
                  s_f = [ V(base + δ_f) − V(base) ] / δ_f

                  Linear P&L approximation:
                     ΔV ≈ Σ_f s_f · shock_f
```

Toy example (`pnl_attribution_drill.py` portfolio):

```
base = Market()         # rate_bump=0, spread_bump=0, vol_shock=0, fx_shock=0
v0   = PF.value(base)

s = (V(rate_bump=1)        − v0,         ← DV01 per bp     (some negative number)
     V(spread_bump=1)      − v0,         ← CS01 per bp
     (V(vol_shock=0.01)    − v0)/0.01,   ← vega per 1.00 vol point
     (V(fx_shock=0.01)     − v0)/0.01)   ← FX-Δ per 1.0 fx shock

# Used as the building block of the parametric VaR σ = √(sᵀΣs) = 2.5327
```

Canonical Python:

```
from dataclasses import replace

def first_order_sensitivities(pf, base):
    """Standard 4-factor sensitivity vector. Bump units: 1bp, 1bp, 0.01, 0.01."""
    v0 = pf.value(base)
    return (
        pf.value(replace(base, rate_bump_bp   = base.rate_bump_bp   + 1)) - v0,
        pf.value(replace(base, spread_bump_bp = base.spread_bump_bp + 1)) - v0,
       (pf.value(replace(base, vol_shock      = base.vol_shock      + 0.01)) - v0) / 0.01,
       (pf.value(replace(base, fx_shock       = base.fx_shock       + 0.01)) - v0) / 0.01,
    )
```

Intuition / pattern: **bump-and-revalue ONE factor at a time**. Cost: 1 + #factors evaluations per snapshot. The unit choice (1bp / 0.01 / 0.01) is by convention so the resulting `s_f` numbers are O(book size) — easy to read. **All units must round-trip with Σ's units** when feeding into parametric VaR.

---

14. Second-order sensitivities

> **TL;DR:** `g_f = [V(+δ) − 2V₀ + V(−δ)] / δ²` (central diff); cross via 4-point. Adding to Taylor explain shrinks unexplained ~4×.

What: convexity / curvature. The first-order Taylor `ΔV ≈ s·Δf` misses the `½·g·Δf²` term — material for options, callable bonds, MBS. Computed by **central difference**.

How it's calculated:

```
g_f = [ V(base + δ_f) − 2·V(base) + V(base − δ_f) ] / δ_f²
```

Cross-gamma:

```
g_{f₁,f₂} = [ V(+δ₁,+δ₂) − V(+δ₁,−δ₂) − V(−δ₁,+δ₂) + V(−δ₁,−δ₂) ] / (4·δ₁·δ₂)
```

Formula:

```
                  g_f = [ V(+δ) − 2 V₀ + V(−δ) ] / δ²        (own-gamma)
                  g_{f₁,f₂} ↑ above                          (cross-gamma)

                  ΔV ≈ Σ_f s_f·shock_f  +  ½ Σ_f g_f·shock_f²
                       + Σ_{f₁<f₂} g_{f₁,f₂}·shock_{f₁}·shock_{f₂}
```

Sensitivity	What it captures
Convexity	rate-rate own-gamma (always ≥ 0 for vanilla bonds)
Option gamma	spot-spot own-gamma (positive long-option; cushions losses)
Vanna	spot-vol cross-gamma (smile/skew sensitivity)
Volga / vomma	vol-vol own-gamma (long vega convexity)

Toy example (`pnl_attribution_drill.py` portfolio):

```
gammas: (rate_bp², spread_bp², vol², fx²)
      = (1.855e-4, 4.287e-5, −1.0636, 412.92)

scenario: +15bp rate, +10bp spread, +2 vol pts, 0 fx
second_order = ½·1.855e-4·225 + ½·4.287e-5·100 + ½·(−1.064)·0.0004 + ½·412.92·0
             = +0.02087     + +0.00214      + −0.000213       + 0
             = +0.02280
```

Adding second-order to the explain shrinks unexplained from +0.0304 → +0.0076 (~4× tighter).

Canonical Python:

```
from dataclasses import replace

def own_gammas(pf, base):
    v0 = pf.value(base)
    bump = lambda **kw: pf.value(replace(base, **kw))
    return (
        (bump(rate_bump_bp   = +1) - 2*v0 + bump(rate_bump_bp   = -1)) / 1**2,
        (bump(spread_bump_bp = +1) - 2*v0 + bump(spread_bump_bp = -1)) / 1**2,
        (bump(vol_shock      = +0.01) - 2*v0 + bump(vol_shock   = -0.01)) / 0.01**2,
        (bump(fx_shock       = +0.01) - 2*v0 + bump(fx_shock    = -0.01)) / 0.01**2,
    )

def cross_gamma(pf, base, key_a, delta_a, key_b, delta_b):
    """Cross-gamma between two factors via 4-point central difference."""
    def b(da, db):  return pf.value(replace(base, **{key_a: getattr(base, key_a)+da,
                                                     key_b: getattr(base, key_b)+db}))
    return (b(+delta_a, +delta_b) - b(+delta_a, -delta_b)
          - b(-delta_a, +delta_b) + b(-delta_a, -delta_b)) / (4 * delta_a * delta_b)
```

Intuition / pattern: cost = `2·#factors` evaluations for own-gammas, `4·#pairs` for cross-gammas. For 10 factors: 20 + 180 = 200 evaluations vs 11 for first-order alone. **Most banks compute own-gammas always, cross-gammas only for known-coupled pairs** (rate-vol, spot-vol, equity-credit) because the combinatorial blow-up is otherwise prohibitive.

Sign warnings: - `g_vol` for a long-option book can be NEGATIVE (e.g. forward-vol structures, calendar spreads). - `g_fx` can be huge (412 in the toy) — driven by the FX option's gamma. Compare to first-order FX-Δ to see relative scale.

---

15. Key Rate Duration (KRD)

> **TL;DR:** Per-pillar DV01 vector (2y/5y/10y/30y). Parallel DV01 = Σ KRD. Hedge the VECTOR; canonical pension/LDI failure ignores shape.

What: parallel DV01 hides curve-shape risk. KRD splits DV01 into **per-bucket** sensitivities: 1y bump, 2y, 5y, 10y, 30y — each bumped INDEPENDENTLY by 1bp, others held flat (then linearly interpolated). Sum of KRDs ≈ parallel DV01.

How it's calculated:

```
For each pillar tenor T_k:
   bump z(T_k) by +1bp, leave other pillars flat (linearly interpolate between)
   KRD_k = V(bumped) − V(base)
```

Formula:

```
                KRD_k = V(bump z(T_k) by +1bp) − V(base)

                Σ_k KRD_k  ≈  parallel DV01
                Curve P&L  =  Σ_k KRD_k · Δz_k
```

Pillar	Hedges
2y	front-end (Fed/BoE policy expectations)
5y	belly (most liquid IRS tenor)
10y	benchmark (LDI hedges, MBS duration)
30y	long end (pension liabilities)

Toy example:

```
10y pension liability:  parallel DV01 = $100k/bp
KRD profile:           (2y, 5y, 10y, 30y) = ($0, $5k, $90k, $5k)

Hedge with 5y IRS only (parallel-neutral) — kills the parallel DV01 to ~$0
but leaves +$85k of 10y exposure (and a hedging −$80k of 5y) → STEEPENER risk:
the parallel DV01 of the hedged book is zero by construction, but its KRDs are NOT zero —
a 10y rise / 5y unchanged loses $85k while parallel DV01 says zero.

Correct hedge: ladder of 5y/10y/30y IRS matching ALL FOUR KRDs.
```

Canonical Python:

```python
import numpy as np
from dataclasses import replace

def krd(pf, base, curve, pillars=(2, 5, 10, 30), bump_bp=1.0):
    """Per-pillar key-rate duration via tent-shaped bumps."""
    krds = []
    v0 = pf.value(base)
    for k in pillars:
        bumped_curve = curve.copy()
        bumped_curve[k] += bump_bp / 1e4
        new_base = replace(base, curve=bumped_curve)
        krds.append(pf.value(new_base) - v0)
    return np.array(krds)
```

Intuition / pattern: parallel DV01 is a **scalar**; KRD is a **vector**. Hedging the scalar is parallel-neutral but **shape-naive**; hedging the vector requires one liquid instrument per pillar. **Pension fund LDI failure mode**: bought 5y IRS to neutralise total DV01 on a 30y liability → parallel-immune, drowned by steepener.

---

16. Computation methods

TL;DR: Forward FD (1 reval); central FD (2; $O(\delta^2)$); AAD (~3-5× one fwd pass for ALL Greeks). AAD pays off above ~50 factors.

Method	Cost (per sensitivity)	Pros	Cons
Forward FD	1 reval	cheapest first-order	$O(\delta)$ bias
Central FD	2 reval	$O(\delta^2)$ bias; gold standard	2× cost
Bump-and-revalue (full)	1 + #factors	exact for the actual pricer	slow for big books
AAD (adjoint)	~3-5× one fwd pass	O(1) for ALL sensitivities	requires AD-instrumented pricer
Pathwise / likelihood-ratio	similar to MC	Greeks alongside price for MC	variance can be high

Bump-size selection

Optimal `δ` balances **truncation error** ($\propto \delta$) against **roundoff** ($\propto \varepsilon/\delta$ where $\varepsilon \sim 1e{-}16$):

```
δ* ≈ (ε)^(1/2) · scale_of_factor    (forward FD)     ≈ 1e-8 for unit-scale factors
δ* ≈ (ε)^(1/3) · scale_of_factor    (central FD)     ≈ 1e-5
```

In practice for finance: **1bp** for rates/spreads, **1% (0.01)** for FX, **1 vol pt (0.01)** for vol — chosen by convention for human readability, not optimality. Roundoff is rarely the binding constraint when book value is O(millions).

Adjoint Algorithmic Differentiation (AAD)

Idea: a pricer with N factors can be augmented to compute ALL N first-order Greeks in a single backward pass that costs ~3-5× the forward valuation. Cf. `tape recording` then `backward propagation` — same machinery as deep-learning autograd.

When to use: high-dimensional Greeks (FRTB SBM has hundreds of buckets), or vectorisable MC pricers where the adjoint can run on GPU. **Cost of adoption**: pricer must be re-implemented with AD-compatible primitives (`jax`, `pytorch`, internal C++ AD library).

Canonical Python (jax sketch — shows the idea, not production):

```
import jax
import jax.numpy as jnp

def bond_price(curve, cfs, times):
    return jnp.sum(cfs * jnp.exp(-curve * times))

# All Greeks in a single AD call
greeks = jax.grad(bond_price)(curve_array, cfs_array, times_array)
# greeks is a vector ∂V/∂curve, one entry per pillar — KRDs for FREE
```

Pattern: bump-and-revalue is **embarrassingly parallel** (independent per factor) — most banks vectorise it before adopting AAD. AAD pays off when factor count > ~50 or when you need higher-order Greeks at scale.

---

17. Black / Black-Scholes Greeks (closed-form)

> **TL;DR:** Closed-form Δ, Γ, ν, Θ, ρ from Black-76 d_1/d_2. **Γ uses F (forward), NOT S (spot)**, in denominator.

What: when the pricer IS closed-form Black/BS, you don't need finite-difference — write down `∂V/∂x` analytically. Faster and exact (no roundoff).

Black on forward F (Black-76; same as IRS swaptions, FRA options, commodity futures options):

Greek	Call	Put
Price	`DF(T)·[F·N(d₁) − K·N(d₂)]`	`DF(T)·[K·N(−d₂) − F·N(−d₁)]`
Δ (delta)	`DF(T)·N(d₁)`	`−DF(T)·N(−d₁)`
Γ (gamma)	`DF(T)·φ(d₁) / (F·σ·√T)`	same as call
ν (vega)	`DF(T)·F·φ(d₁)·√T`	same as call
Θ (theta)	`−DF(T)·[F·φ(d₁)·σ/(2√T) + r·F·N(d₁) − r·K·N(d₂)]`	analogous
ρ (rho)	`−T·V_call` (Black-76 convention)	`−T·V_put`

where $d_1 = [\ln(F/K) + \frac{1}{2}\sigma^2 T] / (\sigma\sqrt{T})$, $d_2 = d_1 - \sigma\sqrt{T}$.

Convention pitfall: **gamma uses F (forward), not S (spot)**, in the denominator. The `treasury_products_card.html` Greeks table had this wrong in a previous draft.

Toy example:

```
F = 0.045, K = 0.042, σ = 0.20, T = 2.0, DF(T) = 0.91576

d₁ = [ln(0.045/0.042) + ½·0.04·2] / (0.20·√2) = (0.0690 + 0.04)/0.2828 = 0.3855
d₂ = 0.3855 − 0.2828 = 0.1027

V_call = 0.91576 · [0.045·N(0.3855) − 0.042·N(0.1027)]
       = 0.91576 · [0.045·0.6501 − 0.042·0.5409]
       = 0.91576 · [0.02925 − 0.02272]
       = 0.00598       (option value per unit notional)

Δ = 0.91576 · 0.6501 = 0.5953
Γ = 0.91576 · φ(0.3855) / (0.045·0.20·√2)
  = 0.91576 · 0.3708 / 0.01273
  = 26.69                         (large because F is tiny)
ν = 0.91576 · 0.045 · 0.3708 · √2 = 0.02161
```

Canonical Python:

```python
import numpy as np
from scipy.stats import norm

def black76(F: float, K: float, sigma: float, T: float, DF: float,
            is_call: bool = True) -> dict:
    """Black-76 price and Greeks on a forward F. Returns dict with V, delta, gamma, vega, theta, rho."""
    v_sqrtT = sigma * np.sqrt(T)
    d1 = (np.log(F / K) + 0.5 * sigma**2 * T) / v_sqrtT
    d2 = d1 - v_sqrtT
    if is_call:
        V     = DF * (F * norm.cdf(d1) - K * norm.cdf(d2))
        delta = DF * norm.cdf(d1)
    else:
        V     = DF * (K * norm.cdf(-d2) - F * norm.cdf(-d1))
        delta = -DF * norm.cdf(-d1)
    gamma = DF * norm.pdf(d1) / (F * v_sqrtT)
    vega  = DF * F * norm.pdf(d1) * np.sqrt(T)
    return dict(V=V, delta=delta, gamma=gamma, vega=vega)
```

Intuition / pattern: closed-form Greeks are **free** once you have the price — no extra revaluations. They are also the SOURCE for FRTB SBM bucket sensitivities. **Sanity check**: at-the-money (F=K), Δ ≈ 0.5·DF(T), Γ peaks, Vega peaks. Far OTM/ITM, Δ → 0/DF(T), Γ → 0.

Part IV — P&L Attribution

The valuation-desk daily question: "Yesterday's P&L was $−3M. **Why?**" Attribution decomposes the realised P&L into per-factor contributions plus an unexplained residual. Same residual is what FRTB's **PLA test** checks to decide whether your internal model can be used for capital.

```
Full P&L      = V(today's market) − V(yesterday's market)         (the truth)
Explained     = Σ_f  s_f · Δf  +  ½ Σ_f g_f · Δf²  +  cross-gamma   (Taylor)
Unexplained   = Full − Explained                                    (residual)
```

The smaller the residual relative to Full, the better the model.

18. Greek-based Taylor explain

TL;DR: `ΔV ≈ Σ s·Δf + ½Σ g·Δf²`; residual = unexplained. Rule: |unexp| < 5% |full| for vanilla derivs.

What: decompose the realised P&L into per-factor first-order (delta) + second-order (gamma) contributions using the **Taylor expansion** of the pricer.

How it's calculated:

```
1. Compute today's full-reval P&L:  ΔV_full = V(today) − V(yesterday)
2. Compute sensitivities s_f, g_f at YESTERDAY's market.
3. Observe today's factor changes:  Δf₁, …, Δf_N
4. Decompose:
      first_order_f  = s_f · Δf
      second_order_f = ½ · g_f · Δf²
      first_total    = Σ_f first_order_f
      second_total   = Σ_f second_order_f
      unexplained    = ΔV_full − first_total − second_total
```

Formula:

```
                  ΔV ≈ Σ_f s_f·Δf  +  ½ Σ_f g_f·Δf²  +  Σ_{f<g} g_{f,g}·Δf·Δg

                  Per-factor attribution:
                       first_f  = s_f · Δf
                       second_f = ½ · g_f · Δf²
                  Residual ("unexplained") = ΔV_full − (Σ first + Σ second)
```

Toy example (`pnl_attribution_drill.py`, scenario = +15bp rate, +10bp spread, +2 vol pts, 0 fx):

```
First-order:
   rate    = s_rate   · 15        = −2.9377
   spread  = s_spread · 10        = −1.2261
   vol     = s_vol    · 0.02      = +1.1095
   fx      = s_fx     · 0         = +0.0000
   first_total                    = −3.0543

Second-order:
   ½ · 1.855e-4 · 15²  = +0.02087
   ½ · 4.287e-5 · 10²  = +0.00214
   ½ · (−1.064) · 0.02² = −0.00021
   ½ · 412.92 · 0²     = +0.00000
   second_total        = +0.02281

Full reval P&L           = −3.0239
Explained                = −3.0543 + 0.0228 = −3.0315

Unexplained (first only) =  −3.0239 − (−3.0543) = +0.0304
Unexplained (first+second) = −3.0239 − (−3.0315) = +0.0076    ← ~4× tighter
```

The residual `+0.0076` is **cross-gamma + higher-order**. For a near-linear book, this stays tiny; for an options-heavy book, you must include cross-gammas to stay within FRTB PLA bands.

Canonical Python:

```python
from dataclasses import astuple

def attribute_pnl(pf, base, scenario, sens, gammas):
    """Greek-based explain: returns dict of per-factor first, second, full, unexplained."""
    v0 = pf.value(base)
    v1 = pf.value(scenario.apply(base))
    shocks = astuple(scenario)
    first  = [s * d for s, d in zip(sens, shocks)]
    second = [0.5 * g * d**2 for g, d in zip(gammas, shocks)]
    full = v1 - v0
    return {
        "rate": first[0], "spread": first[1], "vol": first[2], "fx": first[3],
        "first_order":  sum(first),
        "second_order": sum(second),
        "full":         full,
        "unexplained":  full - sum(first) - sum(second),
    }
```

Intuition / pattern: **P1 with Taylor expansion against the realised shock**. The four numbers everyone wants on the daily explain sheet are: **per-factor first-order contributions**, **second-order (gamma)**, **full P&L**, **unexplained residual**. The order they appear in is the order of explanatory power.

Standard sanity check: `|unexplained| < 10% · |full|` is the rule of thumb for a linear book; **< 5%** with gamma is typical for vanilla derivatives.

---

19. Risk-factor explain (RFE)

> **TL;DR:** Sens RFE (order-indep, misses cross); full RFE (cross-residual); sequential RFE (no residual, order-dep). Pick per use-case.

What: same idea as Taylor explain but **decomposes against THE ACTUAL factor moves** in a multi-factor world. Each component answers: "if ONLY this factor had moved, what P&L would have resulted?" using either sensitivity-based or **full-reval-of-one-factor**.

How it's calculated:

```
1. Sensitivity flavour (P1):  contribution_f = s_f · Δf      (same as §18 first-order)

2. Full-reval flavour (P2):  contribution_f = V(scenario with ONLY Δf applied) − V(base)
                              cross-effects bucketed into a separate "interaction" residual.

3. Step-by-step (sequential):
     V₀ = V(base)
     V₁ = V(base with Δf₁)                →  contribution_1 = V₁ − V₀
     V₂ = V(base with Δf₁, Δf₂)           →  contribution_2 = V₂ − V₁
     …
     V_N = V(base + all shocks) = V(today)
   → ORDER-DEPENDENT (Σ contributions = full P&L exactly, but per-factor split depends on order).
```

Formula:

```
                    Sensitivity RFE:  contribution_f = s_f · Δf
                    Full-reval RFE:   contribution_f = V(only Δf) − V(base)
                    Sequential RFE:   apply shocks IN ORDER; differences
                                      → Σ = full, order-dependent
```

Toy example (full-reval RFE on the drill 12 scenario):

```
V₀ = V(base) = (some baseline value V0)
Apply ONLY rate +15bp:   V_r = V(rate_only) → rate contribution = V_r − V₀
Apply ONLY spread +10bp: V_s = V(spread_only) → spread contribution = V_s − V₀
Apply ONLY vol +0.02:    V_v = V(vol_only)   → vol contribution = V_v − V₀

Sum of single-factor contributions:  ≈ −3.0315  (close to but not equal to full −3.0239)
Difference = "interaction" residual = cross-gamma + higher-order terms
```

Canonical Python:

```python
from dataclasses import replace

def rfe_fullreval(pf, base, shock_dict: dict) -> dict:
    """Full-reval RFE: each factor applied alone vs base; residual = cross effects."""
    v0 = pf.value(base)
    contributions = {}
    for factor, delta in shock_dict.items():
        scenario_one = replace(base, **{factor: getattr(base, factor) + delta})
        contributions[factor] = pf.value(scenario_one) - v0
    all_shocks = replace(base, **{k: getattr(base, k) + v for k, v in shock_dict.items()})
    contributions["interaction"] = (pf.value(all_shocks) - v0
                                    - sum(contributions.values()))
    contributions["full"] = pf.value(all_shocks) - v0
    return contributions
```

Intuition / pattern: **sensitivity RFE is fast (P1) and order-independent but misses cross-effects**. **Full-reval RFE is slower (P2) but per-factor numbers reflect actual non-linearity** — at the cost of an interaction residual that doesn't reduce nicely. **Sequential RFE eliminates the residual but introduces order-dependence** (which factor goes first?), so it's used for reporting ("regulator: please use this order"), not analysis.

Choose: - Linear-ish book → **sensitivity RFE** (cheap, accurate). - Options book → **full-reval RFE** + separate cross-gamma report. - Regulatory submission → **sequential** (no residual, fixed order).

---

20. Unexplained / hypothetical / actual P&L

TL;DR: Actual = book truth; Hyp = SOD-positions on EOD-market; RT = same with VaR pricer. Two residuals: intraday (Actual−Hyp), PLA (Hyp−RT).

The three P&L numbers every trading desk reports daily:

Name	Definition	Use
Actual P&L	end-of-day book value − start-of-day book value (incl. new trades, intraday hedges)	accounting truth
Hypothetical P&L	end-of-day book revalued on TODAY's market, but using start-of-day positions	for backtesting VaR — what a frozen book lost
Risk-Theoretical P&L	start-of-day positions, end-of-day market, computed with VaR-engine pricer	for FRTB PLA — what VaR's pricer predicts
Unexplained P&L	Actual − Hypothetical (= intraday trading / fees / new-deal MTM)	desk attribution

Formula:

```
Actual_PnL    = V_eod(positions_eod, market_eod)
                − V_sod(positions_sod, market_sod)

Hyp_PnL       = V(positions_sod, market_eod)
                − V(positions_sod, market_sod)

RT_PnL (FRTB) = V_VaR_pricer(positions_sod, market_eod)
                − V_VaR_pricer(positions_sod, market_sod)

Unexplained   = Actual − Hyp     (new deals, intraday)
PLA-residual  = Hyp    − RT      (model-vs-front-office)
```

Toy example:

```
Start-of-day book V_sod      = 100.00
End-of-day actual V_eod      =  97.30      (Actual P&L = −2.70)
Same SOD positions revalued on EOD market =  97.50  (Hyp P&L = −2.50)
VaR-engine pricer on same:                =  97.65  (RT  P&L = −2.35)

Unexplained_intraday = −2.70 − (−2.50) = −0.20   ← intraday trading / fees
PLA_residual         = −2.50 − (−2.35) = −0.15   ← front-office vs VaR model
```

Canonical Python (skeleton):

```
def actual_vs_hyp_vs_rt(positions_sod, positions_eod, market_sod, market_eod,
                        front_office_pricer, var_engine_pricer):
    actual = front_office_pricer(positions_eod, market_eod) - front_office_pricer(positions_sod, market_sod)
    hyp    = front_office_pricer(positions_sod, market_eod) - front_office_pricer(positions_sod, market_sod)
    rt     = var_engine_pricer  (positions_sod, market_eod) - var_engine_pricer  (positions_sod, market_sod)
    return {
        "actual": actual, "hyp": hyp, "rt": rt,
        "unexplained_intraday": actual - hyp,
        "pla_residual":         hyp    - rt,
    }
```

Intuition / pattern: **two completely different residuals to track**. Unexplained_intraday is about **desk behaviour** (did the desk trade in/out of positions today?); PLA_residual is about **model quality** (does VaR's pricer agree with the front-office pricer?). Both should be small; the second is what FRTB's PLA test (§21) actually examines.

---

21. FRTB PLA test (Spearman + KS)

> **TL;DR:** Spearman ρ ≥ 0.80 + KS < 0.09 = GREEN. Worst zone wins. Red → desk excluded from IMA → SA capital.

What: regulator's pass/fail on whether the VaR engine's P&L matches the front-office (hypothetical) P&L closely enough over the last 12 months. **Two statistics, two thresholds**.

How it's calculated:

```
Over a 12-month window (≈250 days), collect (Hyp_PnL_t, RT_PnL_t) pairs.

Statistic 1:  Spearman rank correlation ρ_s between the two series.
                ρ_s ≥ 0.80   →  green
                0.70 ≤ ρ_s < 0.80  →  amber
                ρ_s < 0.70   →  red

Statistic 2:  Kolmogorov-Smirnov distance KS = max_x | F_Hyp(x) − F_RT(x) |
                KS < 0.09    →  green
                0.09 ≤ KS < 0.12  →  amber
                KS ≥ 0.12    →  red

Joint outcome:  WORST colour of the two.    Red → desk excluded from IMA → SA capital.
```

Formula:

```
                  ρ_s   = corr( rank(Hyp_PnL), rank(RT_PnL) )
                  KS    = sup_x | F_Hyp(x) − F_RT(x) |

                  Joint colour = worst({ρ_s zone, KS zone})
```

Toy example:

```
Over 250 days collect (Hyp, RT) pairs.
ρ_s = 0.83  →  green
KS  = 0.07  →  green
Desk: green → eligible for IMA capital.

If model misprices a vol-of-vol exposure:  ρ_s could stay 0.85 (overall direction OK)
but KS = 0.15 (the tail distribution diverges) → RED → SA fallback.
```

Canonical Python:

```
import numpy as np
from scipy.stats import spearmanr, ks_2samp

def frtb_pla(hyp_pnl: np.ndarray, rt_pnl: np.ndarray) -> dict:
    """Return Spearman + KS + zone colours per FRTB CRE 32."""
    rho, _ = spearmanr(hyp_pnl, rt_pnl)
    ks, _  = ks_2samp(hyp_pnl, rt_pnl)
    def spearman_zone(r):
        if r >= 0.80: return "GREEN"
        if r >= 0.70: return "AMBER"
        return "RED"
    def ks_zone(k):
        if k < 0.09: return "GREEN"
        if k < 0.12: return "AMBER"
        return "RED"
    order = {"GREEN": 0, "AMBER": 1, "RED": 2}
    zones = (spearman_zone(rho), ks_zone(ks))
    joint = max(zones, key=order.get)
    return {"spearman": rho, "ks": ks, "spearman_zone": zones[0],
            "ks_zone": zones[1], "joint_zone": joint}
```

Intuition / pattern: **two complementary distance metrics**. Spearman catches "are the rankings aligned?" — i.e., does the model rank scenarios in the same order as front-office? KS catches "are the distributions the same shape?" — i.e., does the tail match? You can pass one and fail the other.

Common failure modes: - Missing risk factor in VaR engine (e.g. forgot basis spread) → KS fails (tail shape diverges). - Different vol surface treatment → Spearman fails (rankings drift). - Pricer differences for exotics → both fail.

---

22. Carry / pull-to-par / theta

> **TL;DR:** Pure time-effect P&L (no market move). Carry = expected daily; market move = surprise. Separate column on daily explain.

What: even with zero market move, a book makes/loses P&L from **time passing**: carry (running yield), pull-to-par (accrued discount/premium), theta (option time decay). Reported daily as a separate explain component.

How it's calculated:

```
Carry_t      ≈ coupon accrual per day  +  funding cost (repo / OIS)
Pull-to-par  ≈ (par − price) · (1/years_remaining) per day   [for bonds]
Theta        = ∂V/∂t · 1 day                                  [for options]

Total time-effect P&L = V(t+1, same market) − V(t, same market)
```

Formula:

```
                  Time_PnL  =  V(market_today, t = today + 1day)
                             − V(market_today, t = today)

                  Decomposes as:
                       carry + pull-to-par + theta + cross-roll
```

Component	Sign for long position
Coupon carry	+ (you receive coupons)
Funding (repo) carry	− (you pay financing)
Pull-to-par (discount bond)	+ (price drifts up to par)
Pull-to-par (premium bond)	− (price drifts down to par)
Option theta	− (time decay; you pay for vol)

Toy example:

```
5y 4% bond, price 92.07 (discount bond).
Carry  = 4 · (1/365)            ≈ +0.0110 / day      (coupon accrual)
Pull-to-par ≈ (100 − 92.07)/5 · (1/365) ≈ +0.0043 / day

Long 1Y ATM call, vega = 5, σ=0.20, T=1.
Theta ≈ −V · 1/(2·T)·(1day/365) ≈ very small per day, accelerates near expiry.
```

Canonical Python:

```python
def time_effect_pnl(pf, base, days_forward: int = 1):
    """Pure time-effect (no market move). Requires pricer with explicit valuation date."""
    base_t   = base                          # valuation date = today
    base_t1  = base.advance_date(days_forward)  # valuation date = today + 1d
    return pf.value(base_t1) - pf.value(base_t)
```

Intuition / pattern: **separate "what would have happened anyway" from "what moved"**. Carry is the **expected** daily P&L; market-driven P&L is the **surprise**. A desk's Sharpe ratio is driven by getting carry > funding, not by market timing — for many fixed-income desks, weeks of patient carry get wiped out by one bad market day.

Reporting convention: time-effect appears as a separate column on the daily P&L sheet so the trader can see at a glance: `Full = Time + Market_first_order + Market_second_order + Unexplained`.

Part V — Estimation Inputs

VaR engines, sensitivity multipliers, and capital all depend on **estimated** vol, covariance, and correlation. Garbage in → garbage out. This part covers the canonical estimators and their failure modes.

23. Historical (realised) volatility

TL;DR: Rolling-window sample std. Equal weights → old shocks have same weight as today's; stale during regime shifts.

What: simple sample standard deviation of returns over a rolling window.

How it's calculated:

```
returns_t = ln(P_t / P_{t-1})        (log returns; additive across time)
σ̂window = std(returns_t over last N days, ddof=1)
σ̂annual = σ̂daily · √252              (annualise; 252 trading days)
```

Formula:

$$\sigma^2_t = \frac{1}{N-1} \cdot \sum_{i=t-N+1}^{t} (r_i - \bar{r})^2$$
$$\hat{\sigma}_{annual} = \hat{\sigma}_{daily} \cdot \sqrt{252}$$

Choice	Default	Why
`N` window	60–250 days	trade-off: short = responsive, long = stable
log vs simple returns	log	additive across time; nearly equal for `|r|<5%`
`ddof`	1	unbiased (Bessel correction)

Toy example:

```
Daily returns r = [+0.5%, −0.7%, +0.3%, …] over N=60 days.
mean ≈ 0; std = 0.012 → σ̂daily = 1.2%
σ̂annual = 0.012 · √252 = 19.05%
```

Canonical Python:

```python
import numpy as np

def realised_vol(prices: np.ndarray, window: int = 60, annualise: bool = True) -> float:
    """Rolling-window realised vol. Returns annualised by default."""
    r = np.diff(np.log(prices))
    sigma_daily = r[-window:].std(ddof=1)
    return sigma_daily * np.sqrt(252) if annualise else sigma_daily
```

Intuition / pattern: **equally-weighted average over the window**. Old observations have the same weight as today's → a single shock 200 days ago inflates today's vol the same as a shock yesterday. Solved by EWMA (§24).

---

24. EWMA (RiskMetrics)

TL;DR: $\sigma^2_t = \lambda \cdot \sigma^2_{t-1} + (1-\lambda) \cdot r^2_{t-1}$. RiskMetrics λ=0.94 daily; half-life 11.2d. One parameter; no mean reversion.

What: exponentially-weighted vol. Each new return updates `σ²` by a convex combination with `(1 − λ)` weight, giving an EXPONENTIAL decay on past observations.

How it's calculated:

```
σ²_t = λ · σ²_{t-1}  +  (1 − λ) · r²_{t-1}
       memory of past         yesterday's shock contribution
                          (RiskMetrics: forecast for t MADE at t-1)
```

Formula:

```
                  σ²_t = λ · σ²_{t-1} + (1 − λ) · r²_{t-1}

                  RiskMetrics standard:  λ = 0.94  (daily)
                                         λ = 0.97  (monthly)
```

Half-life and effective N:

```
half-life = ln(0.5) / ln(λ)     # days for a weight to halve
At λ = 0.94:  half-life = 11.2 days,  effective N ≈ 33 days.
```

Toy example:

```
σ²_{t-1} = 0.0001 (σ=1%), today r_t = +2% (a big day).
σ²_t = 0.94 · 0.0001 + 0.06 · (0.02)² = 0.0000940 + 0.0000240 = 0.0001180
σ̂_t  = 1.086%  (modest update — only 6% weight on today's shock)
```

Canonical Python:

```
import numpy as np

def ewma_vol(returns: np.ndarray, lam: float = 0.94) -> np.ndarray:
    """EWMA conditional vol series. Returns σ̂t (not σ²)."""
    var = np.zeros_like(returns)
    var[0] = returns[0]**2
    for t in range(1, len(returns)):
        var[t] = lam * var[t-1] + (1 - lam) * returns[t-1]**2
    return np.sqrt(var)
```

Intuition / pattern: **a single tunable parameter `λ`** controls memory length. RiskMetrics' default `λ=0.94` was estimated by JP Morgan in 1996 across major asset classes — broadly OK but **not optimal for every series**. GARCH (§25) generalises by letting both the past-vol weight AND the shock-weight be fitted.

Pitfall: EWMA assumes returns are zero-mean. Subtract the mean if `|μ| > 0.5σ`, otherwise leave it.

---

25. GARCH(1,1)

TL;DR: `σ²_t = ω + α·r²_{t-1} + β·σ²_{t-1}`. Adds mean reversion to σ²_∞ = ω/(1−α−β). Forecast: `σ̂_{t+h}² = σ²_∞ + (α+β)^h · (σ²_t − σ²_∞)`.

What: vol depends on yesterday's vol AND yesterday's squared return, with each having a fitted weight and a long-run constant. Captures **mean-reversion** in vol that EWMA misses.

How it's calculated:

```
σ²_t = ω  +  α · r²_{t-1}  +  β · σ²_{t-1}

α + β < 1  →  vol mean-reverts to long-run level σ²_∞ = ω / (1 − α − β)
α + β = 1  →  IGARCH (integrated GARCH, equivalent to EWMA with λ = β)
α + β > 1  →  non-stationary; refuse the fit.

Fit: maximum likelihood under Normal (or t) innovations.  Library: `arch`.
```

Formula:

```
                  σ²_t = ω + α · r²_{t-1} + β · σ²_{t-1}

                  σ²_∞ = ω / (1 − α − β)        (long-run variance)
                  half-life = ln(0.5) / ln(α+β)
```

Toy example (typical equity fit):

```
ω = 1e-6,  α = 0.08,  β = 0.90      (α+β = 0.98 → high persistence)
σ²_∞ = 1e-6 / 0.02 = 5e-5  →  σ_∞ = 0.707% daily ≈ 11.2% annual
half-life = ln(0.5) / ln(0.98) = 34.3 days
```

Multi-step forecast (this is the punchline that EWMA cannot replicate):

```
σ²_{t+h | t}  =  σ²_∞  +  (α + β)^h · (σ²_t − σ²_∞)

As h → ∞:  σ²_{t+h}  →  σ²_∞  (mean reversion).
At h=10 with α+β=0.98:  weight on (σ²_t − σ²_∞) is 0.98^10 = 0.817
                          → most of today's vol shock survives 10 days out.
EWMA: σ²_{t+h} = σ²_t for all h (no reversion). Wrong for long-dated VaR scaling.
```

Canonical Python:

```python
from arch import arch_model         # uv add arch
import numpy as np

def garch11_fit(returns: np.ndarray) -> dict:
    """Fit GARCH(1,1) with Normal innovations. Returns params + conditional vol series."""
    am  = arch_model(returns * 100, mean="Zero", vol="GARCH", p=1, q=1, dist="normal")
    res = am.fit(disp="off")
    omega, alpha, beta = res.params["omega"], res.params["alpha[1]"], res.params["beta[1]"]
    return {
        "omega": omega, "alpha": alpha, "beta": beta,
        "sigma_long_run": np.sqrt(omega / (1 - alpha - beta)) / 100,
        "half_life":      np.log(0.5) / np.log(alpha + beta),
        "sigma_t":        res.conditional_volatility / 100,
    }
```

Intuition / pattern: **three parameters generalising EWMA**. The big improvement over EWMA: vol forecasts mean-revert. `σ^{t+h}` converges to `σ_∞` as `h → ∞`, with rate `(α+β)^h`. EWMA's forecast stays constant at `σ^t` for any horizon — wrong for long-dated VaR.

Extensions: - **GJR-GARCH** (`vol="GARCH", o=1`): asymmetric — negative returns ramp up vol more than positive (leverage effect). - **EGARCH**: log-variance → no positivity constraint, captures asymmetry. - **Student-t innovations** (`dist="t"`): fattens tails.

25b. DCC-GARCH + tail-dependence copulas

> **TL;DR:** DCC = time-varying correlations; t-copula = non-zero tail dependence. Closes the 'diversification fails in crisis' gap that Gaussian copula leaves.

What: univariate GARCH (§25) gives **time-varying volatility** but assumes correlations are constant. **DCC-GARCH** (Engle 2002) lets the correlation matrix `R_t` evolve over time — captures the "correlations → 1 in stress" effect that destroys diversification when you need it most. **Copulas** (Gaussian, Student-t, Archimedean) model the **dependence structure** separately from marginals, capturing **tail dependence** that linear correlation misses.

How DCC is calculated:

```
1. Fit univariate GARCH(1,1) per asset → σ̂i,t and standardised residuals z_i,t = r_i,t / σ̂i,t.
2. Q_t = (1 − α − β) · Q̄  +  α · z_{t-1} z_{t-1}ᵀ  +  β · Q_{t-1}
         long-run target          shock              memory
3. R_t = diag(Q_t)^{−½} · Q_t · diag(Q_t)^{−½}        (normalise to correlation)
4. Σ_t = D_t · R_t · D_t  where D_t = diag(σ̂i,t)
```

Copula models (dependence ⊥ marginals via Sklar's theorem):

```
F(x_1, …, x_n) = C( F_1(x_1), …, F_n(x_n) )    (joint CDF = copula of marginal CDFs)

Gaussian copula:    C(u) = Φ_R(Φ⁻¹(u_1), …, Φ⁻¹(u_n))    NO tail dependence.
Student-t copula:   adds ν degrees-of-freedom → upper AND lower tail dependence ↑ as ν ↓.
Clayton (Archim):   lower tail dependence only (good for credit default).
Gumbel  (Archim):   upper tail dependence only (joint upside surprises).
```

Tail-dependence coefficient:

```
λ_lower = lim_{q→0} P[X_1 ≤ F_1⁻¹(q) | X_2 ≤ F_2⁻¹(q)]
λ_upper = lim_{q→1} P[X_1 > F_1⁻¹(q) | X_2 > F_2⁻¹(q)]

Gaussian: λ_lower = λ_upper = 0   (asymptotic independence — diversification "wins" in extremes — counterfactual!
Student-t(ν): λ_lower = λ_upper = 2 · T_{ν+1}(−√((ν+1)(1−ρ)/(1+ρ)))   (positive for any ν < ∞)
Clayton(θ): λ_lower = 2^{−1/θ},  λ_upper = 0
```

Formula:

```
                  DCC:  R_t evolves; Σ_t = D_t R_t D_t
                  Copula: joint = C(marginals) — Sklar's theorem
                  Student-t / Clayton copulas have λ > 0 in tails
```

Toy example (Gaussian vs t-copula MC VaR):

```
Same marginals (each ~N(0,1)), ρ = 0.5.
1-day VaR(99%) of equal-weight 2-asset portfolio:
   Gaussian copula MC:   VaR ≈ 1.61
   t-copula (ν=4) MC:    VaR ≈ 2.05   (28% higher because tail-dep means joint extremes)
   t-copula (ν=8) MC:    VaR ≈ 1.81
```

Canonical Python (DCC via `mgarch`; copula via `copulas` or `statsmodels`):

```
# DCC: production uses dedicated packages (mgarch, rmgarch (R), bekkpy)
# Sketch — fit per-asset GARCH, then DCC:
from arch import arch_model
import numpy as np

def standardised_residuals(returns: np.ndarray) -> np.ndarray:
    """Per-asset GARCH → standardised innovation series, ready for DCC or copula fit."""
    Z = np.zeros_like(returns)
    for i in range(returns.shape[1]):
        res = arch_model(returns[:, i] * 100, vol="GARCH", p=1, q=1).fit(disp="off")
        Z[:, i] = returns[:, i] / (res.conditional_volatility / 100)
    return Z

# Student-t copula calibration:
from scipy.stats import t, kendalltau
def t_copula_calibrate(Z: np.ndarray, nu_grid=(3, 4, 5, 6, 8, 12)) -> tuple[float, np.ndarray]:
    """Coarse t-copula calibration: rank-correlation matrix + ν via likelihood grid."""
    U = (Z.argsort(axis=0).argsort(axis=0) + 1) / (Z.shape[0] + 1)  # rank to (0,1)
    R = np.corrcoef(t.ppf(U, df=nu_grid[0]), rowvar=False)
    # full implementation: MLE on (R, ν) — see scikit-learn-extra or `copulas` lib
    return nu_grid[0], R
```

Intuition / pattern: **decouple marginals from dependence**. Standard VaR / mean-variance implicitly use a Gaussian copula (zero tail-dependence) — empirically wrong. Switch to t-copula for **any** equity/credit/multi-asset VaR engine that needs to handle the "everything fell together" days. DCC for time-varying correlation under regime shifts.

When to use: - Cross-asset VaR / capital → t-copula (tail dependence). - Default risk on a portfolio of names → Clayton (lower-tail). - Regime-changing market → DCC-GARCH. - Marginals fat-tailed but dependence Gaussian → calibrate marginals empirically, then plug into Gaussian copula.

---

26. Implied volatility

TL;DR: Solve σ s.t. BS(σ) = market_price. Brent root-find (not Newton) for robustness near boundaries. Vol surface = IV across strikes × expiries.

What: the `σ` that makes a Black-Scholes (or Black-76) price match an observed market option price. **Forward-looking** — embeds the market's view of future vol.

How it's calculated:

```
Solve for σ:    market_price = BS(S, K, T, r, σ, ...)
                                          ▲
Numerical: Newton-Raphson on  f(σ) = BS(σ) − market_price ;  f'(σ) = vega.
Initial guess:  σ_0 = sqrt(2π/T) · |market_price| / S    (Brenner-Subrahmanyam)
```

Formula (the inversion):

```
                  Find σ such that  BS_price(σ) = mkt_price

                  Newton step:  σ_{n+1} = σ_n − (BS(σ_n) − mkt) / vega(σ_n)

                  Convergence: |BS(σ) − mkt| < tol  (typically 1e-8)
```

Toy example:

```
S=100, K=100, T=1, r=0.04, q=0, observed call price = 10.45.
BS at σ=0.20:  9.41   →  too low,   increase σ
BS at σ=0.25:  11.69  →  too high,  decrease σ
BS at σ=0.224: 10.45  ✓   →  IV = 22.4%
```

Canonical Python:

```
import numpy as np
from scipy.optimize import brentq
from scipy.stats import norm

def bs_call(S, K, T, r, sigma, q=0.0):
    d1 = (np.log(S/K) + (r - q + 0.5*sigma**2)*T) / (sigma*np.sqrt(T))
    d2 = d1 - sigma*np.sqrt(T)
    return S*np.exp(-q*T)*norm.cdf(d1) - K*np.exp(-r*T)*norm.cdf(d2)

def implied_vol(market_price, S, K, T, r, q=0.0, is_call=True) -> float:
    """Brent root-find — more robust than Newton at the boundaries."""
    if not is_call:
        f = lambda v: bs_call(S, K, T, r, v, q) - (S - K*np.exp(-r*T)) - market_price  # put via parity
    else:
        f = lambda v: bs_call(S, K, T, r, v, q) - market_price
    return brentq(f, 1e-6, 5.0, xtol=1e-8)
```

Intuition / pattern: **the market's price translated into vol units**. A vol surface (IV across all strikes × expiries) is the canonical "vol object" used by trading desks — far richer than a single realised number. Smile/skew shapes encode tail-risk pricing.

Pitfalls: - Very deep OTM options → very small price → IV can explode numerically. - Newton can fail near the boundary; **use Brent's method** for robustness. - Always check `market_price > intrinsic_value`; otherwise no real IV exists.

---

27. Sample covariance

> **TL;DR:** `cov(R, rowvar=False, ddof=1)`. Gold standard ONLY when T >> N; singular when T < N.

What: per-pair sample covariance of asset returns, assembled into an N×N matrix.

How it's calculated:

```
R: T × N matrix of returns (T days, N assets).
μ̂ = mean(R, axis=0)              (length-N)
Σ̂ = (1/(T−1)) · (R − μ̂)ᵀ (R − μ̂)   (N × N)
```

Formula:

$$\hat{\Sigma}_{ij} = \frac{1}{T-1} \cdot \sum_t (r_{t,i} - \hat{\mu}_i)(r_{t,j} - \hat{\mu}_j)$$

Equivalently: $\hat{\Sigma} = \text{cov}(R, \text{rowvar=False}, \text{ddof=1})$

Toy example:

```
T=250, N=3 assets.
np.cov(R, rowvar=False) → 3×3 symmetric PSD matrix.
diag(Σ̂) = per-asset sample variance.
off-diag = pairwise covariances; correlations = Σ̂ij / (σ_i · σ_j).
```

Canonical Python:

```
import numpy as np

def sample_cov(returns: np.ndarray) -> np.ndarray:
    """returns shape (T, N). Bessel-corrected, PSD by construction (when T > N)."""
    return np.cov(returns, rowvar=False, ddof=1)
```

Intuition / pattern: **gold standard ONLY when T >> N**. When T < N (more assets than days), the sample covariance is **singular** (rank ≤ T−1) → can't invert → can't compute parametric VaR / mean-variance weights. Solved by shrinkage (§28).

Pitfalls: - Time-varying covariance: sample assumes stationary; use EWMA-covariance or DCC-GARCH if vol regimes shift. - Outlier sensitivity: a single crash day in the window inflates all variances. - Look-ahead bias: never include returns from after your evaluation date.

---

28. Ledoit-Wolf shrinkage

> **TL;DR:** Σ_shrink = δ·F + (1−δ)·S, δ fitted to minimise MSE. Regularises when N≈T; standard fix for parametric VaR / mean-variance with limited history.

What: shrink the **noisy sample covariance** toward a **structured target** (constant correlation, identity, or single-factor) by a fitted weight `δ`. Reduces estimation error, makes the matrix invertible even when `T < N`.

How it's calculated:

```
Σ̂shrink = δ · F  +  (1 − δ) · S
                     target    sample
                    matrix     covariance
```

`F` is the **shrinkage target** — most common choices: - **Identity scaled**: `F = (tr(S)/N) · I` (zero correlations, equal variances). - **Constant correlation**: average pairwise correlation → all off-diag get the same `ρ̄`.

`δ` is fitted to **minimise expected MSE** between `Σ̂shrink` and the true Σ; closed-form formula in Ledoit & Wolf (2004).

Formula (Ledoit-Wolf optimal δ, full form):

```
                    δ* = max(0, min(1, (π − ρ) / (T · γ)))

                    π = Σ_{i,j} Var(S_ij)        (sample-cov estimation noise)
                    ρ = Σ_{i,j} Cov(S_ij, F_ij)  (sample-target covariance)
                    γ = ||F − Σ||_F² = Σ_{i,j}(F_ij − E[S_ij])²  (model bias)

                    Identity target ⇒ ρ ≈ 0 ⇒ δ* ≈ π / (T·γ)
```

Toy example:

```
T=60 days, N=100 stocks.  Sample cov has rank ≤ 59 → singular.
Shrink toward (tr(S)/100) · I.   Fitted δ = 0.45.
Σ̂shrink = 0.45 · F + 0.55 · S → full rank, well-conditioned.
```

Canonical Python:

```python
from sklearn.covariance import LedoitWolf  # uv add scikit-learn
import numpy as np

def ledoit_wolf_cov(returns: np.ndarray) -> tuple[np.ndarray, float]:
    """Shrink sample cov toward (tr/N)·I. Returns (Σ_shrunk, shrinkage_intensity δ)."""
    lw = LedoitWolf().fit(returns)
    return lw.covariance_, float(lw.shrinkage_)
```

Intuition / pattern: **regularisation for covariance matrices**. Δ near 0 → trust the sample; Δ near 1 → trust the target. Typical Δ for daily equity factor returns: 0.1–0.4. Always check `eigvals(Σ̂shrink) > 0` after fitting.

When to use: - N (assets) ~ T (days) — sample is noisy. - Need `Σ⁻¹` for mean-variance, parametric VaR, MVaR. - Bayesian-flavoured "blend prior with data" framing.

---

29. PSD repair (eigenvalue clipping)

What: a sample correlation matrix should be **positive semi-definite** (PSD: all eigenvalues ≥ 0). Real data often produces small NEGATIVE eigenvalues (numerical noise, missing data, regulator-mandated bumps). Fix by clipping.

How it's calculated:

```
1. Eigen-decompose:    Σ = Q Λ Qᵀ
2. Clip:               Λ' = diag(max(λ_i, ε))      ε small (e.g. 1e-8)
3. Reconstruct:        Σ_psd = Q Λ' Qᵀ
4. (Optional) Rescale to preserve diagonal:    D = diag(√(diag(Σ)/diag(Σ_psd)))
                                                Σ_final = D Σ_psd D
```

Formula:

```
                  Σ = Q Λ Qᵀ
                  Λ' = diag(max(λ_i, ε))
                  Σ_psd = Q Λ' Qᵀ      (PSD; diagonal may drift)
                  Σ_final = D Σ_psd D  to restore unit diagonal
```

Toy example:

```
3-asset correlation matrix from data:
   [1.00  0.85  0.75]
   [0.85  1.00  0.95]
   [0.75  0.95  1.00]

eigenvalues = [2.692, 0.323, −0.015]    ← tiny negative → NOT PSD
Clip negative to 1e-8 → reconstruct → PSD ✓ with off-diagonals barely changed.
```

Canonical Python:

```
import numpy as np

def nearest_psd(M: np.ndarray, eps: float = 1e-8) -> np.ndarray:
    """Clip negative eigenvalues to eps. Symmetrise. Rescale diagonal to preserve variances."""
    M_sym = 0.5 * (M + M.T)
    vals, vecs = np.linalg.eigh(M_sym)
    vals = np.clip(vals, eps, None)
    M_psd = vecs @ np.diag(vals) @ vecs.T
    d = np.sqrt(np.diag(M) / np.diag(M_psd).clip(eps))
    return d[:, None] * M_psd * d[None, :]
```

Intuition / pattern: **a covariance matrix that isn't PSD breaks downstream code** — `chol` fails, parametric VaR can return imaginary numbers, MVO can produce wild leverage. Clip-and-rescale is the **standard fix**. More principled alternatives: Higham's nearest-correlation-matrix algorithm (iterative projection).

Conditioning warning: clipping to `ε=1e-8` makes the matrix technically PSD but may leave it **ill-conditioned** for `Σ⁻¹` (mean-variance, MVaR). If you need to invert, clip to a higher floor (~`1e-4 · λ_max`) or use Higham.

29b. Liquidity-Adjusted VaR (LVaR)

> **TL;DR:** VaR + ½·|pos|·(μ_BAS + k·σ_BAS) — adds bid-ask cost. Horizon-scaled: √h · VaR_1 for forced multi-day liquidation.

What: standard VaR ignores **transaction cost** and **liquidation horizon**. A position you can't unwind in one day has more risk than the daily-VaR suggests. LVaR adds two corrections: **(a) bid-ask spread cost** and **(b) extended-horizon vol**.

How it's calculated (Bangia-Diebold-Schuermann decomposition):

```
LVaR(α) = VaR(α)  +  ½ · |position| · (μ_BAS + k·σ_BAS)

where:  μ_BAS = mean relative bid-ask spread  (e.g. 0.0010 for 10bp)
        σ_BAS = std of relative bid-ask spread
        k = quantile (e.g. 3 for 99%, normal assumption)

The ½ factor: you only cross HALF the spread on liquidation (mid → bid).
```

Horizon-adjusted LVaR (when h-day liquidation, not 1-day):

```
LVaR_h(α)  =  √h · VaR_1(α)  +  liquidation_cost

For non-trivial gamma: also add ½·Γ·(√h·σ)²  (convexity over the longer horizon).
```

Formula:

```
                  LVaR = VaR_market_risk + LiquidationCost
                       = z(α)·σ + ½·|pos|·(μ_BAS + k·σ_BAS)

                  For h-day forced liquidation:
                  LVaR_h = √h · VaR_1 + LiquidationCost
```

Toy example:

```
Position size = $10M. Asset σ_daily = 1% → standard VaR(99%) = 2.326·1%·$10M = $232.6k.
Bid-ask spread: μ_BAS = 5bp (0.0005),  σ_BAS = 2bp (0.0002), k = 3.
Liquidation cost = ½ · $10M · (0.0005 + 3·0.0002) = ½ · $10M · 0.0011 = $5,500.
LVaR(99%) = $232,600 + $5,500 = $238,100.   (2.4% uplift — modest for liquid asset)

Same position in an emerging-market asset, BAS μ = 50bp, σ = 20bp:
Liquidation cost = ½ · $10M · (0.005 + 3·0.002) = $55,000.   LVaR = $287,600 (24% uplift).
```

Canonical Python:

```python
import numpy as np
from scipy.stats import norm

def lvar_bangia(var_daily: float, position: float,
                bas_mean: float, bas_std: float, conf: float = 0.99) -> float:
    """Bangia-Diebold-Schuermann LVaR. var_daily = market-risk VaR. BAS in decimal (50bp = 0.005)."""
    k = norm.ppf(conf)
    liq_cost = 0.5 * abs(position) * (bas_mean + k * bas_std)
    return var_daily + liq_cost

def lvar_horizon(var_1day: float, h_days: int, liq_cost: float) -> float:
    """Forced-liquidation LVaR scaling. iid-normal assumption — fails under autocorrelation."""
    return np.sqrt(h_days) * var_1day + liq_cost
```

Intuition / pattern: **two distinct frictions to add to VaR**. (a) bid-ask cost is a deterministic levy on the position size — small for liquid futures, huge for illiquid OTC. (b) horizon scaling penalises positions that can't be unwound quickly — FRTB IMA bakes this in via per-factor LH (§37); BDS LVaR is the pre-FRTB shorthand many desks still use as a single overlay.

Limits: assumes the BAS distribution is stationary (false in crises — spreads blow out 10×); doesn't capture **market impact** (selling large size moves the price). For impact: add a `λ·|q|^a` term (Almgren-Chriss).

Part VI — Stress & Scenarios

VaR captures **the distribution of normal-day moves**. Stress testing answers the complementary question: **"what if a specific historical/hypothetical bad day happens?"** — a single deterministic scenario, full-revalued.

30. Historical scenarios

TL;DR: Named events replayed: Black Monday, LTCM, Lehman, COVID, Gilt 2022. Single deterministic shock vector, full-revalue.

What: replay specific named events. Standard library:

Scenario	Period	Key factor moves
Black Monday 1987	1987-10-19	S&P −22%, vol up 200%
Russian default / LTCM	1998-08 to 1998-10	EMBI +800bp, US Treasury 10y −150bp (flight)
9/11 reopen	2001-09-17	S&P −5%, USTs flight, energy +10%
Lehman week	2008-09-15 to 2008-09-19	Libor-OIS +200bp, S&P −12%, USTs −80bp, gold +5%
March 2020 (COVID)	2020-03-09 to 2020-03-23	S&P −34%, VIX 80, USTs −50bp, IG +250bp, HY +600bp
Sept 2022 (gilt LDI crisis)	2022-09-23 to 2022-09-28	30y UK gilt +120bp in 3 days, GBP −5%, LDI funds margin-called

How it's calculated:

```
For each named scenario:
   1. Look up the factor shocks observed during the event.
   2. Full-reval the current book under those shocks.
   3. Report scenario P&L = V(shocked) − V(base).
```

Toy example (Lehman week scenario on a 5y vanilla bond):

```
Lehman week shocks:  rate −80bp,  credit_spread +200bp
Bond DV01 = 0.0425,  CS01 = 0.0425
Linear P&L ≈ (−0.0425)·(−80) + (−0.0425)·(+200) = 3.40 − 8.50 = −5.10
Full-reval P&L (drill 8 portfolio): slightly different due to convexity.
```

Canonical Python:

```
HISTORICAL_SCENARIOS = {
    "lehman_week": {"rate_bp": -80, "spread_bp": +200, "vol_shock": +0.10, "fx_shock": +0.03},
    "covid_march": {"rate_bp": -50, "spread_bp": +250, "vol_shock": +0.40, "fx_shock": -0.05},
    "gilt_2022":   {"rate_bp": +120,"spread_bp": +30,  "vol_shock": +0.20, "fx_shock": -0.05},
}

def scenario_pnl(pf, base, scenario_name: str) -> float:
    shocks = HISTORICAL_SCENARIOS[scenario_name]
    return pf.value(Scenario(**shocks).apply(base)) - pf.value(base)
```

Intuition / pattern: **a single deterministic factor vector applied as a shock**. Doesn't replace VaR — complements it. VaR says "1 in 100 days I lose at least $X"; scenario says "if Lehman happens again, I lose $Y, EXACTLY".

Calibration sources: scenario shocks should be measured from the event window (peak-to-trough for risk-off, day-of-shock for fast moves). Don't average over the recovery — that diluteies the shock.

31. Hypothetical scenarios

> **TL;DR:** Forward-looking templates (parallel shift, steepener, blowout, vol spike). Solvency II SCR is a prescribed library of these.

What: forward-looking shocks based on **plausible but unobserved** events. Used for what-if analysis, capital planning, ICAAP/Solvency II reverse stress.

Common templates:

Scenario	Description	Standard shocks
Parallel shift	yield curve ±100bp	rate_bp = ±100, others flat
Steepener / flattener	2s10s	2y +25bp, 10y +50bp / 2y +50bp, 10y +25bp
Bull/bear flattener	rate AND direction	bull flat = 2y unchanged, 10y −50bp; bear flat = 2y +50bp, 10y unchanged
Credit blowout	IG +200bp / HY +500bp	spread_bp by credit bucket
Vol spike	implied vol +50% relative	vol_shock = 0.5 · base_vol
FX 3σ	DXY ±10%	fx_shock = ±0.10
Combined risk-off	rate −50bp, IG +100bp, vol +30%, equity −20%	multi-factor

Canonical Python:

```
HYPOTHETICAL_SCENARIOS = {
    "parallel_+100":   {"rate_bp": 100},
    "parallel_-100":   {"rate_bp": -100},
    "bear_steepener":  {"rate_2y_bp": 25, "rate_10y_bp": 50},
    "bull_flattener":  {"rate_2y_bp": 0,  "rate_10y_bp": -50},
    "vol_spike_50pct": {"vol_shock": 0.10},     # absolute uplift assuming base σ ≈ 20%
    "risk_off":        {"rate_bp": -50, "spread_bp": 100, "vol_shock": 0.06, "fx_shock": -0.05},
}

# apply via the same scenario_pnl(pf, base, name) as historical
```

Intuition / pattern: **stress isn't VaR**. VaR is statistical; stress is **stylised**. Use both. The Solvency II SCR for market risk uses **prescribed 1-in-200-year hypothetical shocks** (rate ±62bp at 1y, equity −39%, spread ±100bp for AAA → +750bp for unrated, FX ±25%) — calibrated to historical 99.5% but applied as deterministic stresses.

32. Reverse stress test

> **TL;DR:** Find the cheapest (Mahalanobis-minimal) factor shock producing target loss L*. ICAAP/Pillar 2 requirement.

What: instead of "what's my P&L given this scenario?", ask **"what scenario produces a P&L of $−X?"** where X = critical loss threshold (e.g. regulatory capital, going-concern boundary).

How it's calculated:

```
Choose target loss L*. Find a scenario s* such that:
   1. PnL(s*) = −L*       (loss equals target)
   2. s* is "plausible" (e.g., Mahalanobis distance ≤ k)
   3. s* minimises distance to current market state

Solve:
   minimise  sᵀ Σ⁻¹ s            (Mahalanobis distance²)
   subject to  PnL(s) = −L*
```

For a linear book: closed-form via Lagrange.

```
λ = −L* / (sensᵀ · Σ · sens)             (Lagrange multiplier; negative for a LOSS)
s* = λ · Σ · sens                          (shock vector)
Mahalanobis dist = √(s*ᵀ Σ⁻¹ s*) = L* / σ_pnl
```

Toy example (drill 13 portfolio, find scenario that loses 10):

```
σ_pnl = 2.5327, sens = [s_rate, s_spread, s_vol, s_fx]
λ = −10 / σ_pnl² = −10 / 6.414 = −1.559
s* = −1.559 · Σ · sens

(numerically): roughly proportional to the "worst-direction" eigenvector of Σ × sens.
Mahalanobis dist = 10 / 2.5327 = 3.95σ  ←  ~3.95-sigma event in factor space.
```

Canonical Python (linear, closed-form):

```python
import numpy as np

def reverse_stress_linear(target_loss: float, sens: np.ndarray, cov: np.ndarray):
    """Find the smallest (Mahalanobis) factor shock that produces target_loss for a LINEAR book."""
    pnl_var = sens @ cov @ sens
    lam = -target_loss / pnl_var
    s_star = lam * (cov @ sens)
    mahalanobis = np.sqrt(s_star @ np.linalg.inv(cov) @ s_star)
    return s_star, mahalanobis
```

Intuition / pattern: **"find the cheapest way to lose $X"**. The answer points to the **factor combination you're most exposed to**: a hedged book has a HIGH Mahalanobis distance for a given loss (you'd need a freak combination of shocks); a concentrated book has a LOW one (one obvious bad scenario destroys you).

Regulatory use: PRA / Fed / EBA all require reverse stress as part of ICAAP. Banks must publish: "what scenario takes us below CET1 minimum?" The plausibility threshold is qualitative — defended in writing.

---

33. Aggregating scenario P&L

> **TL;DR:** Report the WORST scenario + the full TABLE. DON'T sum/average — double-counts shared shocks (Lehman + COVID both contain equity drop).

Problem: you run 20 different scenarios. How do you summarise?

Approaches:

```
1.  WORST SCENARIO:           max over scenarios of |PnL|.       Single number.
2.  CORRELATED SUM:            Σ_s w_s · |PnL_s|                  Weighted by likelihood.
3.  DASHBOARD (no aggregation): table of all 20 scenario P&Ls.    Just look at it.
```

Aggregation pitfall: scenarios are NOT independent draws from a distribution. Adding their losses **double-counts shared shocks** — Lehman week and COVID both contain a large equity drop; sum implies the equity shock happens twice. **Don't average / sum** scenario P&Ls; **report the table**.

Reporting convention (typical for trading desk):

Scenario	P&L ($)	Main driver
Lehman week	−5.10	credit spread +200bp
COVID March	−7.85	vol +40% on long-gamma book
Gilt 2022	+1.20	short-duration book benefits from rate rise
Parallel +100	−2.45	linear DV01
Bear steepener	−1.85	KRD-10y > KRD-2y
Worst loss	**−7.85**	COVID March

Why averaging double-counts: many scenarios share factor shocks. Lehman (S&P −12%) and COVID (S&P −34%) both load on the equity shock; adding their P&Ls implies the equity move happens TWICE. Same for the credit-spread move present in both Lehman and COVID. Sum-of-scenarios overstates true tail loss by 50-200% on a multi-asset book.

Intuition / pattern: **the worst-case number is the summary; the table is the analysis**. Senior management gets the worst number; the trading desk gets the table to know which scenario is biting and why.

Part VII — Capital Frameworks

Regulatory market-risk capital sits on top of all the methods above. Two frameworks live in parallel: **Basel SA-CCR** (counterparty exposure) and **FRTB** (market-risk capital, replacing Basel 2.5 IMA + SA from January 2025).

34. Basel SA-CCR

> **TL;DR:** `EAD = 1.4 · (RC + PFE)`. Replaces CEM. Counterparty exposure for OTC + SFTs; feeds CCR-RWA.

What: Standardised Approach for Counterparty Credit Risk — replaces CEM (Current Exposure Method). Computes **EAD** (Exposure at Default) for OTC derivatives netting sets and SFTs.

How it's calculated:

```
EAD = α · (RC + PFE)            α = 1.4 (regulatory multiplier)
RC  = max(V − C, 0)              Replacement Cost; V = MtM, C = net collateral
PFE = aggregate add-on per asset class × multiplier

multiplier = min(1, floor + (1 − floor) · exp((V − C) / (2 · aggregate_add_on)))
```

Formula:

```
                    EAD = 1.4 · ( RC + PFE )

                    RC  = max(V − C, 0)
                    PFE = mult · Σ_AssetClass AddOn_AC
                    mult = min(1, 0.05 + 0.95·exp((V−C)/(2·Σ AddOn)))
```

Toy example:

```
Net MtM V = $10M;  collateral C = $5M  →  RC = $5M
IR add-on aggregated = $20M;  FX add-on = $5M  →  Σ = $25M
multiplier = min(1, 0.05 + 0.95·exp(5/50)) = min(1, 0.05 + 0.95·1.105) = 1.0
PFE = 1.0 · 25 = $25M
EAD = 1.4 · (5 + 25) = $42M
```

Canonical Python (illustrative — production has dozens of edge cases):

```python
import math

def sa_ccr_ead(mtm: float, collateral: float, addons: dict[str, float]) -> float:
    """Simplified SA-CCR EAD. addons is per asset-class aggregated add-on ($)."""
    rc = max(mtm - collateral, 0)
    total_addon = sum(addons.values())
    mult = min(1.0, 0.05 + 0.95 * math.exp((mtm - collateral) / (2 * max(total_addon, 1e-6))))
    pfe = mult * total_addon
    return 1.4 * (rc + pfe)
```

Intuition / pattern: **EAD ≈ today's MtM + future drift**. Replaces the very crude "notional × supervisory factor" of CEM with **asset-class add-ons sensitive to maturity, delta, and supervisory factors**. EAD then feeds the **counterparty risk-weighted asset (RWA)** calc.

---

35. Basel IMA (legacy)

> **TL;DR:** MRC = max(VaR_t, k·avg$_{60}$VaR) + max(sVaR_t, k·avg$_{60}$sVaR). Phased out by FRTB Jan 2025.

What: Basel II.5 Internal Models Approach — the old VaR-based market-risk capital, phased out by FRTB IMA in 2025 but still relevant for legacy systems and emerging-market regulators.

How it's calculated:

```
Market_Risk_Charge = max(VaR_t, k_VaR · VaR_avg_60d) + max(sVaR_t, k_sVaR · sVaR_avg_60d)
                       general VaR (99%, 10-day)              stressed VaR (II.5 addition)

k_VaR, k_sVaR ∈ [3.0, 4.0] (traffic-light, §11)
```

Formula:

```
                  MRC_legacy = max(VaR_t, k·avg₆₀(VaR)) + max(sVaR_t, k·avg₆₀(sVaR))
                  Plus: IRC (incremental risk charge) + CRM (comprehensive risk)
```

Toy example:

```
VaR_t = $50M, avg60_VaR = $48M, k_VaR = 3.0 (green)  →  charge = max(50, 144) = $144M
sVaR_t = $80M, avg60_sVaR = $75M, k_sVaR = 3.0  →  charge = max(80, 225) = $225M
MRC_legacy = $144M + $225M = $369M
```

Intuition / pattern: **VaR is the foundation; sVaR forces memory of 2008; multiplier penalises poor backtests**. Replaced because: (a) VaR not coherent, (b) tail beyond 99% unmeasured, (c) capital can fall during calm markets even if balance-sheet risk hasn't.

---

36. FRTB SA (Sensitivities-Based Method)

> **TL;DR:** TWO-level sensitivity aggregation: within-bucket K_b, across-bucket Delta. Three correlation scenarios (low/med/high), take max.

What: Standardised Approach under Fundamental Review of the Trading Book. Sensitivity-based — bank computes **delta, vega, curvature** per regulator-defined bucket, applies prescribed risk weights and correlations, sums to capital.

Three components (two-level aggregation per risk class):

```
SBM Capital = Σ_RiskClass [Delta + Vega + Curvature]

Delta:
  WITHIN bucket b:  K_b = √( Σ_i WS_i² + Σ_{i≠j} ρ_ij · WS_i · WS_j )
                    WS_i = RW_i · s_i  (risk-weighted sensitivity per factor in bucket b)
                    ρ_ij = WITHIN-bucket correlation (prescribed)
  ACROSS buckets:   Delta = √( Σ_b K_b² + Σ_{b≠c} γ_bc · S_b · S_c )
                    S_b = Σ_i WS_i  (signed sum over factors in bucket b)
                    γ_bc = ACROSS-bucket correlation (prescribed)

Vega:       Same TWO-LEVEL structure on vega sensitivities; buckets per maturity × moneyness.

Curvature:  CVR_k_up   = −[ V(+RW · risk-factor shock) − V(base) − RW · delta ]   (LOSS, signed +)
            CVR_k_down = −[ V(−RW · risk-factor shock) − V(base) + RW · delta ]
            K_b_curv = √( Σ_b max(CVR_up_b, CVR_down_b)² ) within bucket; then across-bucket
            aggregation analogous to Delta with curvature correlations.

THREE CORRELATION SCENARIOS: banks compute Delta/Vega/Curvature under low (× 0.75),
medium (× 1.00), and high (× 1.25) correlation scaling — take MAX across scenarios.
```

Plus: Default Risk Charge (DRC), Residual Risk Add-On (RRAO).

Risk classes:

Class	Buckets	Typical bucket dim
GIRR (interest rate)	5 currencies × 10 tenors	50
CSR (credit spread)	sectors × ratings × tenors	~300
Equity	sectors × cap	13
FX	currency pairs	~150
Commodity	sub-types	11

Toy example (illustrative — full calc has 1000s of buckets):

```
GIRR delta in USD, 5y bucket: WS = 2.5% · DV01_5y × 1e4    (RW 2.5% for USD-5y)
GIRR delta in USD, 10y bucket: WS = 2.4% · DV01_10y × 1e4
Within-currency aggregation:  √(WS₅² + WS₁₀² + 2·ρ·WS₅·WS₁₀)
                                                  ↑
                                        ρ ≈ 0.99 for adjacent tenors

Sum across currencies and add other risk classes for total SBM charge.
```

Canonical Python (single-bucket pattern, scales to all):

```python
import numpy as np

def sbm_delta_charge(weighted_sens: np.ndarray, corr_matrix: np.ndarray) -> float:
    """Within-bucket aggregation. weighted_sens shape (n_factors,); corr (n,n)."""
    # FRTB allows negative interior when correlation is negative → take real part of √
    inner = weighted_sens @ corr_matrix @ weighted_sens
    return float(np.sqrt(max(inner, 0)))
```

Intuition / pattern: **a giant sensitivity-aggregation tree**. Three nested levels: 1. Per-factor risk-weighted sensitivity. 2. Within-bucket aggregation via correlation. 3. Across-bucket aggregation via inter-bucket correlation.

The capital number is **always positive** (variance-style formula) but the per-bucket pre-aggregation can be signed. Banks game this by **concentrating risk where correlations work in their favour** (intra-bucket diversification benefit).

---

37. FRTB IMA (ES + SES + NMRF)

> **TL;DR:** ES at α=0.975 with liquidity-horizon scaling. IMCC + SES (NMRF) + DRC. Most banks dropped IMA for many desks due to operational cost.

What: Internal Models Approach under FRTB. **Replaces VaR with ES at 97.5%** on a stressed period with **liquidity-horizon scaling**, plus SES (Stressed ES at desk level) plus NMRF (Non-Modellable Risk Factor add-on).

How it's calculated:

```
IMCC = ρ · ES_F + (1 − ρ) · ES_RS
       where ES_F  = ES on FULL set of modellable factors, scaled by liq-horizon
             ES_RS = ES on REDUCED set (factors with enough data), scaled
             ρ = 0.5 (regulator-fixed)

SES = Σ over each NMRF: stress-scenario loss (similar to historical sVaR per NMRF)

Total IMA charge = IMCC + SES + DRC
```

Liquidity-horizon scaling (FRTB CRE 33.21):

```
The book has factors at different liquidity horizons LH_j ∈ {10, 20, 40, 60, 120} days.
For each LH_j, define S_j = SUBSET of factors with LH ≥ LH_j (so S_j shrinks as j grows).

ES_liq² = ES_T(all_factors)²
         + Σ_{j ≥ 2}  [ ES_T(only_factors_in_S_j) · √( (LH_j − LH_{j-1}) / T ) ]²

where T = base horizon = 10 days. As LH_j grows, FEWER factors are shocked
(only the illiquid ones), so ES_T(S_j) shrinks — the formula adds the
incremental risk from holding the less-liquid factors for longer.

Per-factor LH (FRTB):  rates / FX (10d), equity vol (20d), small-cap equity (60d),
                       credit IG (40-60d), credit HY (120d).
```

Formula:

```
                    IMA = IMCC + SES + DRC
                    IMCC = 0.5·ES_full_stressed + 0.5·ES_reduced_stressed
                    SES  = Σ_NMRF stress_scenario_loss(NMRF)
                    All ES computed at 97.5% on a stressed window
```

Toy example:

```
Desk has $50M of ES at 97.5% on liquid factors.  Liquidity-horizon scaling lifts to $90M.
Reduced-set ES = $75M.   IMCC = 0.5·90 + 0.5·75 = $82.5M.
Plus SES for 3 NMRFs at $5M each = $15M.   Plus DRC = $20M.
Desk IMA capital = $82.5 + $15 + $20 = $117.5M.

Compare to Basel II.5: VaR-based total maybe $90M.   FRTB ~30% higher for the same book.
```

Canonical Python (high-level scaffold; full implementation is 1000s of lines):

```python
import numpy as np

def frtb_ima_charge(es_full: float, es_reduced: float,
                    nmrf_charges: list[float], drc: float, rho: float = 0.5) -> float:
    imcc = rho * es_full + (1 - rho) * es_reduced
    ses  = sum(nmrf_charges)
    return imcc + ses + drc

def liquidity_horizon_scaling(es_full_t10: float, es_subsets: list[tuple[int, float]],
                              base_horizon_days: int = 10) -> float:
    """FRTB CRE 33.21 LH scaling.
    es_full_t10:    ES at base horizon T=10 on the FULL factor set.
    es_subsets:     [(LH_j, ES_T(S_j))] for j=2..J — ES computed at base T but on the
                     SUBSET of factors whose LH ≥ LH_j (drop the liquid ones).
    Returns ES_liq under the non-overlapping accumulation rule.
    """
    total_sq = es_full_t10 ** 2
    prev_lh = base_horizon_days
    for lh_j, es_sj in es_subsets:
        weight = ((lh_j - prev_lh) / base_horizon_days) ** 0.5
        total_sq += (es_sj * weight) ** 2
        prev_lh = lh_j
    return float(total_sq ** 0.5)
```

Intuition / pattern: **ES at 97.5% is the new VaR at 99%** (similar tail mass for Normal: ES(97.5) ≈ VaR(99) under Normal P&L). The expensive bits are: 1. **Backtesting at the DESK level** (not just bank) → desks fail → fall back to SA per desk. 2. **NMRF identification** — every factor without 24 obs/year + 100 obs in window is NMRF → punitive SES. 3. **PLA test** (§21) — every desk must pass quarterly or lose IMA eligibility.

Most banks have backed off IMA for many desks because of the operational cost — by far the largest practical impact of FRTB.

---

38. IRC / DRC (default risk)

> **TL;DR:** Capital for default/migration JUMPS that VaR's Normal misses. 99.9% / 1y. Hammers concentrated single-name positions.

What: capital for default and migration risk on bonds, CDS, single-name credit derivatives. **Outside the VaR framework** because default is a rare-event jump, not a normal-style fluctuation.

IRC (Incremental Risk Charge, Basel II.5):

```
Defines a 99.9% confidence, 1-year capital charge for default + rating migration on trading-book
credit positions.  Typically computed via Monte Carlo on a Vasicek-style ASRF model.
Capital = 99.9% VaR of 1-year credit-loss distribution.
```

DRC (Default Risk Charge, FRTB):

```
Same idea but redefined: SBM-bucket-by-bucket default risk, with:
- LGD ∈ {prescribed values per seniority}
- PDs from external ratings, floored
- Hedging recognition only within the same name (not basket)

DRC = Σ_buckets max( Σ_long JTD_w, Σ_short JTD_w · (−recovery%) )
      JTD_w = jump-to-default weighted exposure
```

Formula (DRC, sketch):

```
                    JTD_i = LGD_i · notional_i − MtM_i
                    WtS_i = w_i · JTD_i        (risk-weighted)
                    Bucket Charge = max(Σ longs, |Σ shorts|) - hedges
                    DRC = Σ_buckets bucket_charge
```

Toy example:

```
$100M long IG corp bond, LGD 60%, MtM par.
JTD = 60% · 100 − 100 = −40        (you LOSE 40 if it defaults; netting in cap)
With LGD 60% bonds yielded above par, JTD captures negative carry if recovery > 100−MtM.
DRC charge ≈ Σ within bucket of JTDs, no diversification across IG bucket.
```

Intuition / pattern: **default is a jump; VaR's Normal assumption misses it entirely**. IRC/DRC adds a one-year, 99.9% charge for **discontinuous credit events**. Strongly punishes concentrated single-name positions (CDS notionals are weighted at full LGD). MBS, CDOs go to a separate charge (CRM, then under FRTB the **securitisation framework**).

---

39. xVA stack (CVA / DVA / FVA / KVA)

> **TL;DR:** Valuation adjustments to risk-free MtM: counterparty credit (CVA), own credit (DVA), funding (FVA), capital (KVA), margin (MVA). Hedged separately by an XVA desk.

What: **valuation adjustments** for OTC derivatives. The "risk-free" pricer is wrong — counterparty default risk (CVA), own default (DVA), funding cost (FVA), and capital cost (KVA) each adjust the MtM. Sum forms the **xVA stack**, charged daily and hedged separately.

Definitions:

```
CVA — Credit Valuation Adjustment
       = expected loss from COUNTERPARTY default over life of trade
       = LGD_cp · ∫₀ᵀ EPE(t) · PD_cp(t) · DF(t) dt
       EPE(t) = E[max(V_t − Collateral_t, 0)]    (Expected Positive Exposure)

DVA — Debit Valuation Adjustment  (the mirror of CVA from the OTHER side)
       = LGD_self · ∫₀ᵀ ENE(t) · PD_self(t) · DF(t) dt
       ENE(t) = E[max(Collateral_t − V_t, 0)]    (Expected NEGATIVE Exposure)
       Accounting controversy: own-default credit is a "profit" → IFRS 13 requires it but
       prudential regulators (Basel) exclude it from capital.

FVA — Funding Valuation Adjustment
       = cost of funding uncollateralised exposures (between OIS and bank's funding rate)
       = ∫₀ᵀ (funding_spread) · E[Exposure(t)] · DF(t) dt
       Hull-White / Burgard-Kjaer FVA framework; charged on uncollateralised legs.

KVA — Capital Valuation Adjustment
       = present value of regulatory capital cost (CCR + CVA capital + market risk) over trade life
       = ∫₀ᵀ (hurdle_rate × required_capital_t) · DF(t) dt
       Charged to compensate shareholders for capital tied up by the trade.

MVA — Margin Valuation Adjustment
       = cost of posting initial margin under SIMM / CCP rules
       = ∫₀ᵀ (margin_funding_spread × IM_t) · DF(t) dt
```

Formula (the stack):

```
                  V_xVA  =  V_risk_free  −  CVA  +  DVA
                                        −  FVA  −  KVA  −  MVA

                  All terms POSITIVE values; signs make the bank's
                  "fair" price LOWER than risk-free (typically).
```

Toy example:

```
10y IRS, V_risk_free = $500k (in bank's favour).
Counterparty: BB-rated, LGD 60%, peak EPE $5M, 5% cumulative PD over 10y.
   CVA ≈ 0.60 · $5M · 0.05 · 0.80 (avg DF) = $120k

Own credit: A-rated, LGD 60%, peak ENE $0.5M, 1% PD over 10y.
   DVA ≈ 0.60 · $0.5M · 0.01 · 0.80 = $2.4k

Funding: bank's funding spread 50bp over OIS, avg uncoll exposure $2M, 10y.
   FVA ≈ 0.0050 · $2M · 6 (PV of annuity) ≈ $60k

Regulatory capital tied up ~ $200k initial; hurdle 10%; 10y.
   KVA ≈ 0.10 · $200k · 6 = $120k

V_xVA = 500 − 120 + 2.4 − 60 − 120 = $202k    (60% reduction from risk-free MtM)
```

Canonical Python (illustrative — production has thousands of lines per xVA):

```python
import numpy as np

def cva(epe_grid: np.ndarray, pd_grid: np.ndarray, df_grid: np.ndarray, lgd: float = 0.6) -> float:
    """Discretised CVA: Σ LGD · EPE(t_i) · ΔPD(t_i) · DF(t_i)."""
    delta_pd = np.diff(np.concatenate(([0], pd_grid)))
    return float(lgd * np.sum(epe_grid * delta_pd * df_grid))

def fva(exposure_grid: np.ndarray, funding_spread: float,
        df_grid: np.ndarray, dt: float) -> float:
    """Discretised FVA: Σ funding_spread · Exposure(t_i) · DF(t_i) · dt."""
    return float(funding_spread * np.sum(exposure_grid * df_grid) * dt)
```

Intuition / pattern: **xVA is the difference between textbook MtM and what the trade actually nets to the bank**. Computed centrally by an **XVA desk** that hedges each component separately: - CVA:

hedged with single-name CDS or index CDS (CDX/iTraxx). - DVA: arguably un-hedgable (banks can't trade their own credit cheaply) — controversial. - FVA: hedged in the bank's funding desk. - KVA: warehoused; charged forward to client.

FRTB-CVA framework (replacing Basel III CVA capital): bank must compute CVA under prescribed risk factors and run the same sensitivity-based aggregation as FRTB SBM (§36) — **CVA becomes part of the market-risk capital number**, not separate.

Sign-trap reminder:

```
Bank's bid (buyer) price:  V_xVA from BANK's perspective
                            = V_riskfree − CVA + DVA − FVA − KVA   (bank pays LESS to compensate)
Bank's ask (seller) price: V_xVA from CLIENT's perspective (bank sells at higher to compensate)

Practical: bid-ask spread on derivatives EMBEDS the xVA hedging cost.
```

Appendix — Coherence axioms (Artzner et al.)

A risk measure `ρ(X)` (loss-representation: ρ ≥ 0 is bad) is **coherent** if it satisfies four axioms:

Axiom	Name	What it means
(1)	**Monotonicity**	$X \le Y \Rightarrow \rho(X) \ge \rho(Y)$ (a worse loss has higher risk)
(2)	**Translation invariance**	$\rho(X + c) = \rho(X) - c$ (adding cash reduces risk by same amount)
(3)	**Positive homogeneity**	$\rho(\lambda \cdot X) = \lambda \cdot \rho(X)$ for $\lambda \ge 0$ (scaling positions scales risk)
(4)	**Subadditivity**	$\rho(X + Y) \le \rho(X) + \rho(Y)$ (diversification can only help)

Why it matters:

```
VaR FAILS (4) in general.  Counterexample: two independent low-prob big-loss assets;
                            individual VaR each = 0, joint VaR > 0.
                            Implication: VaR can punish diversification — pathological.

ES   SATISFIES all 4.       This is the formal reason FRTB switched VaR → ES.
```

Toy counter-example to VaR subadditivity:

```
Two zero-coupon bonds, each defaults independently with p = 0.04, loss = 1.
Individual VaR(0.95) for each = 0   (default prob < 1−α = 0.05)
Combined: P(both default) = 0.0016, P(at least one default) = 0.0784 > 0.05
   → Combined VaR(0.95) = 1     >    sum of individual VaRs = 0
→ Diversification "increased" VaR by the metric.  VaR violates subadditivity. ✗

ES gives a coherent answer that does NOT punish this diversification.
```

Practical takeaway: **prefer ES for capital, prefer VaR for limits**. Limits desks want a metric that doesn't move under intraday rebalancing (VaR's discreteness helps); capital wants a number that diversifies sensibly (ES's coherence helps).

Risk-factor sign cheatsheet

Reference for the daily explain. Under the convention `s = V(bumped) − V(base)` used throughout this document:

Long position in…	DV01	CS01	IE01	FX-Δ	Vega
Vanilla bond	−	−	0	0	0
Linker	−	0	+	0	0
Foreign bond	−	−	0	+	0
FRN	≈0 (between resets)	−	0	0	0
IRS receive-fixed	−	0	0	0	0
IRS pay-fixed	+	0	0	0	0
ZCIS receive-inflation	≈0 at par	0	+	0	0
FX Spot / Fwd (long foreign)	+ (small)	0	0	+	0
CDS protection BUYER	− (small)	+	0	0	0
CDS protection SELLER	+ (small)	−	0	0	0
Long call (FX, swaption)	small	0	0	+Δ	+
Long put	small	0	0	−Δ	+
Pension annuity liability (short)	+	0	−	0	0

Convention swap (vendor systems): Bloomberg / most risk systems report DV01 as **loss-given-up** (positive for long bonds). For those numbers, **flip every sign in the DV01 column above**.

Greeks aggregation rule: per-instrument Greeks **add linearly** when factors are common. So a portfolio's total DV01 = Σ instrument DV01; total vega = Σ instrument vega (only on instruments sharing the same vol surface).
